

Metatranscriptomic Analysis of Microbial Reads from Colorectal Cancer Patient Biopsies

Alice Hong

2026-04-12

```
{r setup, include=FALSE}the knitr::opts_chunk$set(echo = FALSE, warning = FALSE, message = FALSE, results = 'hide') knitr::opts_knit$set(root.dir = normalizePath("."))
```

Introduction

Background

Characterizing microbial abundance and function from human tissue samples such as biopsies presents substantial challenges for analysis. Unlike stool or cecal samples that contain high microbial biomass, mucosal tissue biopsies contain a low ratio of microbial to host cells, causing sequencing depth to be overwhelmingly dominated by host RNA. Therefore, standard metatranscriptomic workflows optimized for microbe-rich samples are often poorly suited for tissue-based studies. At the same time, tissue-derived samples offer a unique opportunity to simultaneously investigate both host transcriptional responses and the activity of spatially localized microbes at the mucosal interface, where many biologically relevant host-microbe interactions occur. As a result, analytical frameworks must balance efficient recovery of low-abundance microbial transcripts with preservation of host-derived signals that provide critical context for interpreting microbial function in situ.

Marker-based taxonomic classifiers such as MetaPhlAn 4 and mOTUs3 rely using species-specific marker genes, but when microbial reads make up only a small fraction of the sample, there may not be enough of those reads to detect them reliably. A 2024 benchmarking paper demonstrates this by showing that in synthetic samples containing 97% host cells, marker-based methods achieved recall as low as 0.05-0.15, meaning the vast majority of species present went undetected. (Pereira-Marques, 2024) This loss of sensitivity at the taxonomic level then carries on into functional profiling, since tools like HUMAnN 3 depend on accurate species identification to build sample-specific pangenome databases for their nucleotide search step.

To address this, Pereira-Marques et al. proposed a metatranscriptomics strategy specifically tailored for low-biomass samples. (Pereira-Marques, 2024) Their workflow suggests Kraken 2/Bracken (with the confidence threshold raised to 0.05 to reduce false positives while preserving high recall) with HUMAnN 3 for functional profiling. By feeding the more complete Kraken 2/Bracken taxonomic profile into HUMAnN 3, a greater proportion of reads are captured during the nucleotide search step, reducing reliance on the computationally expensive translated search and improving overall detection of microbial gene families.

Overview of original experiment

In their study ‘Biofilms and core pathogens shape the tumor microenvironment and immune phenotype in colorectal cancer’ Kvich et al. investigate the interplay between host tissue and mucosa-associated bacteria in colorectal cancer (CRC) using a dual-RNAseq approach (host) combined with fluorescence in situ hybridization (FISH) microscopy. Mucosal colon biopsies were collected from CRC patients (tumor and paired normal tissue) and healthy controls at Zealand University Hospital, Denmark. FISH was used to visualize bacterial biofilms and to detect *B. fragilis* and *Fusobacterium* spp. directly in tissue sections, while RNA sequencing captured both host and microbial transcripts from the same samples.

On the host side, reads were aligned to the human reference genome and used for differential gene expression analysis (DESeq2), pathway enrichment, and immune cell deconvolution using multiple scoring methods. The host RNA-seq analyses revealed that CRC samples with high bacterial activity displayed a pro-inflammatory transcriptomic profile, with enrichment of interleukin-10 signaling, defensins, and chemokine receptor binding pathways, as well as shifts in immune cell infiltration including increased M2 macrophages, regulatory T-cells, and myeloid dendritic cells.

For the microbial component, trimmed reads were classified with Kraken 2/Bracken to determine bacterial community composition, revealing that CRC tissue harbored higher overall bacterial counts and was enriched for Bacteroidota, Fusobacteriota, and Actinobacteria, with *F. nucleatum* accounting for approximately 79% of Fusobacteria. Additionally, reads that did not map to the human genome were re-aligned to a custom concatenated reference of *B. fragilis* and *F. nucleatum* to identify which genes these species were expressing in the tumor microenvironment. The authors reported the top 20 most highly expressed genes for each species, but noted that most were highly conserved housekeeping genes (e.g., *tuf*, *fusA*, *gap*, *dnaK*, *groL*) shared across many bacteria, making species-specific functional inference not possible.

Justification for re-analysis

Kvich et al.’s analysis of the microbial transcriptome was limited in scope. For functional characterization, unmapped (non-human) reads were aligned to only two representative reference genomes, *B. fragilis* and *F. nucleatum*, rather than being profiled against a comprehensive database. This approach is not standard practice in metatranscriptomics as it risks misattributing reads from closely related species to the two chosen references, and it discards transcriptional information from the broader microbial community entirely. The authors acknowledged that the results were largely uninformative, as the most highly expressed genes were conserved across many bacterial species. Notably, no justification was provided for why a community-wide functional profiling approach was not pursued.

Tools such as HUMAnN 3, particularly when combined with optimized Kraken 2/Bracken as proposed above, are specifically designed for this type of analysis. They provide both community-wide functional profiling and species-resolved pathway analysis simultaneously, enabling a far more comprehensive view of microbial activity in the tumor microenvironment. Given that the Kvich et al. dataset comprises approximately 90 deeply sequenced mucosal biopsy samples from CRC, paired normal, and healthy tissues (the type of low-biomass samples the Pereira-Marques workflow was validated for) there is a clear opportunity to re-analyze this data using standard metatranscriptomic methods. Such a re-analysis could reveal functional contributions from the broader microbial community and species-resolved pathway differences between CRC and healthy tissue that the original two-genome approach could not detect.

Overview of methods

This re-analysis is structured in three stages. First, the Kvich et al. microbial analysis is reproduced on the raw unmapped reads to confirm that our pipeline produces similar results as the published paper. Second, additional pre-processing, including quality trimming and human read decontamination using KneadData, is applied to the same reads. Its effect on sample depth and community composition is benchmarked against the baseline, and low-read-depth samples are excluded. Third, functional profiling with HUMAnN3 is performed on the KneadData-cleaned reads (84 samples); this analysis was not attempted by the original authors.

Dataset

Sequencing data containing human reads were not shared by Kvich et al. due to public health information reasons. Instead, a subset of the deeply sequenced RNA-seq data was shared upon agreement between Lasse Kvich, Blaine Gabriel Fritz, and Carolina Tropini. These reads are unmapped reads that did not align to GRCh38. The upstream pipeline consisted of adapter trimming with Cutadapt, ribosomal RNA removal with SortMeRNA, and alignment to GRCh38 with bwa-mem v0.7.17. The unmapped fraction was provided as 119 paired-end samples, named {sample}_unmapped_R1.fq / {sample}_unmapped_R2.fq (238 files total, read length ~150 bp, sequencer Illumina A00962). Metadata was obtained from Kvich and Fritz as well.

There are three biological groups:

Group	Samples	Description
CRC	La___ samples	Tumor biopsies from CRC patients
Paired normal	La___ samples	Paired normal tissue from the same CRC patients
Healthy control	Lasse___ samples	Mucosal biopsies from healthy individuals

Sample naming: Lasse4--Lasse33 in file names correspond to La4--La33 in clinical metadata (same samples, different naming convention). La125 is a sequencing control absent from the metadata and is excluded from all analyses. All R scripts remap names with `str_replace("^Lasse(\\d+)$", "La\\1")`.

All sample names are listed in `samples_all.txt` (119 samples). After KneadData filtering (Part 2), the 86 retained samples are listed in `samples_filtered.txt`.

Computational Environment

All compute jobs were run on the UBC ARC Sockeye HPC cluster using SLURM. Scripts for Parts 1 and 3a are in `scripts/no_kneaddata/`; scripts for Parts 2 and 3b are in `scripts/kneaddata/`. Each directory is numbered in execution order. Per-sample steps were submitted as SLURM array jobs.

Databases:

Database	Version
Kraken2 standard	k2_standard_20260226
KneadData human reference	T2T-CHM13v2.0 (hg39)
HUMANn3 nucleotide	ChocoPhlAn (full)
HUMANn3 protein	UniRef90

The Kraken2 standard database was downloaded as below:

```
cd /scratch/st-ctropini-1/ahong/kraken2_standard_db
wget -c https://genome-idx.s3.amazonaws.com/kraken/k2_standard_20260226.tar.gz \
-O kraken2_standard.tar.gz
tar -xzf kraken2_standard.tar.gz
```

Part 1: Reproducing the Kvich et al. Microbial Analysis

The unmapped reads were used directly, without additional pre-processing, to reproduce the original microbial analyses. This establishes a baseline to compare with the published results before any methodological changes are introduced.

Scripts: `scripts/no_kneaddata/01_align_bwa.sh` through `07_normalize_bracken.py` Local output: `no_kneaddata/`

1.1 Reference genome preparation

Reference genomes for *Bacteroides fragilis* (GCF_003019295.1) and *Fusobacterium nucleatum* (GCF_016889925.1) were downloaded from NCBI. The FASTA sequences and GTF annotation files were concatenated into a single combined reference, then indexed with bwa.

```
cat refs/GCF_003019295.1/GCF_003019295.1_ASM301929v1_genomic.fna \
    refs/GCF_016889925.1/GCF_016889925.1_ASM1688992v1_genomic.fna \
    > refs/combined/combined_bfrag_fnuc.fna

cat refs/GCF_003019295.1/GCF_003019295.1_ASM301929v1_genomic.gtf \
    refs/GCF_016889925.1/GCF_016889925.1_ASM1688992v1_genomic.gtf \
    > refs/combined/combined_bfrag_fnuc.gtf

bwa index refs/combined/combined_bfrag_fnuc.fna
```

Chromosome assignments in the combined reference:

Chromosome	Organism
NZ_CP069563.1	<i>Bacteroides fragilis</i> (locus tag prefix I6J55_RS)
NZ_CP028101.1	<i>Fusobacterium nucleatum</i> (locus tag prefix C7Y58_RS)

1.2 Alignment to bacterial references (script: 01_align_bwa.sh)

Paired-end reads from all 119 samples were aligned to the combined reference with bwa-mem v0.7.19. SLURM array: 119 tasks, 8 CPUs, 64 GB RAM, 24 h.

```
bwa mem \
    -t 8 \
    -R "@RG\tID:{sample}\tSM:{sample}\tLB:{sample}\tPL:ILLUMINA" \
    refs/combined/combined_bfrag_fnuc.fna \
    input/{sample}_unmapped_R1.fq \
    input/{sample}_unmapped_R2.fq \
    | samtools sort -@ 8 -o output/bam/{sample}.bam

samtools index output/bam/{sample}.bam
```

BAMs are on the server only (/scratch/st-ctropini-1/ahong/bfrag3/output/bam/) and are not included in this repository due to file size.

1.3 Alignment QC (script: 02_flagstat.sh)

samtools flagstat was run per sample; results were aggregated into a MultiQC report. SLURM array: 119 tasks, 2 CPUs, 8 GB RAM, 1 h.

```
# Per sample
samtools flagstat \
    -@ 2 \
    output/bam/{sample}.bam \
    > output/qc/flagstat/{sample}.flagstat

# MultiQC aggregation
multiqc output/qc/flagstat \
    --outdir output/qc/multiqc \
    --filename bwa_alignment_qc
```

Per-sample flagstat files: no_kneaddata/qc/flagstat/ (119 files). MultiQC report: no_kneaddata/qc/multiqc/bwa_alignment_qc

- The report shows that mapping rate to the combined *B. fragilis* + *F. nucleatum* reference is very low, as expected by mapping all unmapped reads to just two species. However, some samples (La70 13.8%,

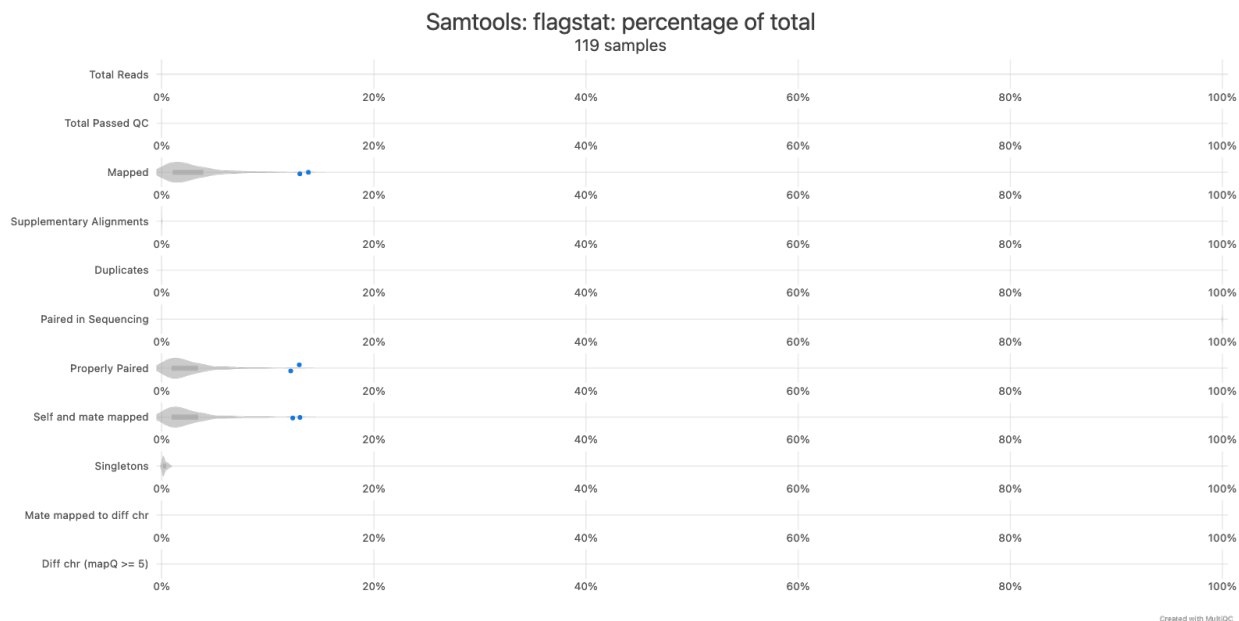


Figure 1: samtools-flagstat-pct-table

La122 12.95, La110 10.64) showed a high percentage. These were also high depth (large number of reads) samples.

- Mean mapping rate is around 2.8%. From the violin plot look appears to be (similar), showing that most mapped pairs do pair properly

1.4 Gene expression quantification (script: 03_featurecounts.sh)

featureCounts quantified per-gene read counts across all 119 BAMs simultaneously. Single SLURM job: 8 CPUs, 32 GB RAM, 24 h.

```
featureCounts \
  -p -B -C \
  -T 8 \
  -a refs/combined/combined_bfrag_fnuc.gtf \
  -F GTF -t CDS -g gene_id \
  -o output/counts/counts.txt \
  output/bam/*.bam
```

Key flags: **-p** paired-end; **-B** requires both mates to map; **-C** discards chimeric pairs; **-t CDS** required because NCBI bacterial GTFs annotate protein-coding genes as CDS features, not **exon**.

Output: no_kneaddata/alignment/counts/counts.txt – gene x sample raw read count matrix.

1.5 Gene annotations (script: 03b_make_gene_annotations.py)

A gene annotation table was generated from the combined GTF by first collecting **product** descriptions from CDS feature lines into a **gene_id** -> **product** dictionary, then joining to each **gene** feature line. This is required because **product** is a CDS-level attribute in NCBI bacterial GTFs, not a gene-level attribute.

```
python scripts/no_kneaddata/03b_make_gene_annotations.py \
  refs/combined/combined_bfrag_fnuc.gtf \
  > no_kneaddata/alignment/gene_annotations.tsv
```

Output: no_kneaddata/alignment/gene_annotations.tsv – columns: gene_id, gene (short name), product.

1.6 Taxonomic profiling – Kraken2 (script: 04_kraken2.sh)

Kraken2 was run on the raw unmapped reads with `--confidence 0.05`, as recommended by Pereira-Marques et al. for low-biomass samples. SLURM array: 119 tasks, 8 CPUs, 100 GB RAM, 24 h. Classified reads discarded (`--output /dev/null`) to save disk space.

```
kraken2 \
  --db /scratch/st-ctropini-1/ahong/kraken2_standard_db \
  --threads 8 --paired --confidence 0.05 \
  --report output/kraken2_conf0.05/{sample}.report \
  --output /dev/null \
  input/{sample}_unmapped_R1.fq input/{sample}_unmapped_R2.fq
```

Output: no_kneaddata/community_composition/kraken2/{sample}.report (119 files).

1.7 Abundance estimation – Bracken (script: 05_bracken.sh)

Bracken redistributes reads assigned to higher taxonomic levels back to species. SLURM array: 119 tasks, 4 CPUs, 16 GB RAM, 4 h.

```
bracken \
  -d /scratch/st-ctropini-1/ahong/kraken2_standard_db \
  -i output/kraken2_conf0.05/{sample}.report \
  -o output/bracken_conf0.05/{sample}.bracken \
  -w output/bracken_conf0.05/{sample}_bracken.report \
  -r 150 -l S
```

Output: no_kneaddata/community_composition/bracken/ – 238 files (`.bracken` + `_bracken.report`, 2 per sample).

1.8 Normalize Bracken counts (script: 07_normalize_bracken.py)

Species-level counts were normalized following Kvich et al.:

1. Read counts per sample from raw R1 FASTQ (`wc -l / 4`)
2. Scaling factor = `min_reads_across_samples / reads_in_sample`
3. Scaled counts = Bracken `new_est_reads` x scaling factor
4. Taxa with `log10(scaled_count + 1) < 0.9` in all samples removed (~8 reads; from Kvich et al. Figure S1)

```
conda activate kraken_env
python scripts/no_kneaddata/07_normalize_bracken.py
```

Output in no_kneaddata/community_composition/normalized/: `counts_unscaled.tsv`, `counts_scaled.tsv`, `counts_filtered.tsv`, `scaling_factors.tsv`.

1.9 Results: Community composition and gene expression

Community composition results are shown for both the raw reads (119 samples, reproducing Kvich et al.) and KneadData-cleaned reads (86 samples) side-by-side. Gene expression results (Table S2, Figure 2J) are based on BWA alignment and featureCounts, which were applied to the raw reads only.

```
BFRAG_CHR <- "NZ_CP069563.1"
FNUC_CHR <- "NZ_CP028101.1"
```

```

type_levels <- c("CRC", "Healthy", "Paired")

# read meta data
meta <- read.delim("meta.tsv") %>%
  mutate(sample_id = as.numeric(str_remove(sample, "[A-Za-z]+")))
paired_meta <- meta %>% filter(type %in% c("Paired", "CRC"))

# Function to fix inconsistent naming (Lasse to La)
rename_lasse <- function(df) {
  colnames(df) <- colnames(df) %>% str_replace("^Lasse(\\d+)$", "La\\1")
  df
}

# Flagstat contains total fragments per sample (read1 line)
parse_flagstat <- function(file) {
  lines <- readLines(file)
  data.frame(sample = tools::file_path_sans_ext(basename(file)),
    total_fragments = as.numeric(str_extract(grep("read1", lines, value =
  ↪ TRUE)[1], "\\d+"))))

flagstat_df <- list.files("no_kneaddata/qc/flagstat", pattern = "\\..flagstat$",
  ↪ full.names = TRUE) %>%
  map_dfr(parse_flagstat) %>%
  mutate(sample = str_replace(sample, "^Lasse(\\d+)$", "La\\1"))

# featureCounts matrix
counts_raw <- read.delim("no_kneaddata/alignment/counts/counts.txt", skip = 1,
  ↪ check.names = FALSE)
colnames(counts_raw) <- colnames(counts_raw) %>%
  str_replace(".*/", "") %>% str_replace("\\.bam$", "") %>%
  str_replace("^Lasse(\\d+)$", "La\\1")
sample_cols <- setdiff(colnames(counts_raw), c("Geneid", "Chr", "Start", "End", "Strand",
  ↪ "Length"))

bfrag_counts <- counts_raw %>% filter(Chr == BFRAG_CHR) %>% select(all_of(sample_cols))
  ↪ %>% colSums()
fnuc_counts <- counts_raw %>% filter(Chr == FNUC_CHR) %>% select(all_of(sample_cols))
  ↪ %>% colSums()

reads_df <- flagstat_df %>%
  left_join(tibble(sample = names(bfrag_counts), bfrag = bfrag_counts), by = "sample")
  ↪ %>%
  left_join(tibble(sample = names(fnuc_counts), fnuc = fnuc_counts), by = "sample")
  ↪ %>%
  mutate(across(c(bfrag, fnuc), replace_na, 0), unmapped = pmax(total_fragments - bfrag -
  ↪ fnuc, 0))

## Warning: There was 1 warning in `mutate()`.
## i In argument: `across(c(bfrag, fnuc), replace_na, 0)`.
## Caused by warning:
## ! The `...` argument of `across()` is deprecated as of dplyr 1.1.0.
## Supply arguments directly to `.fns` through an anonymous function instead.
##
## # Previously

```

```

## across(a:b, mean, na.rm = TRUE)
##
## # Now
## across(a:b, \(x) mean(x, na.rm = TRUE))

# Normalized Bracken counts, as described by authors
counts_scaled <-
  ↪ read.delim("no_kneaddata/community_composition/normalized/counts_scaled.tsv", row.names
  ↪ = 1, check.names = FALSE) %>% rename_lasse()
counts_unscaled <-
  ↪ read.delim("no_kneaddata/community_composition/normalized/counts_unscaled.tsv", row.names
  ↪ = 1, check.names = FALSE) %>% rename_lasse()
keep <- intersect(colnames(counts_scaled), meta$sample)
counts_scaled <- counts_scaled[, keep]
counts_unscaled <- counts_unscaled[, keep]

sf_df <- read.delim("no_kneaddata/community_composition/normalized/scaling_factors.tsv")
  ↪ %>%
  ↪ mutate(sample = str_replace(sample, "^Lasse(\\d+)$", "La\\1"))

# KneadData normalized Bracken counts (86 samples)
counts_scaled_kd <-
  ↪ read.delim("kneaddata/community_composition/normalized/counts_scaled.tsv", row.names
  ↪ = 1, check.names = FALSE)
counts_unscaled_kd <-
  ↪ read.delim("kneaddata/community_composition/normalized/counts_unscaled.tsv",
  ↪ row.names = 1, check.names = FALSE)
keep_kd <- intersect(colnames(counts_scaled_kd), meta$sample)
counts_scaled_kd <- counts_scaled_kd[, keep_kd]
counts_unscaled_kd <- counts_unscaled_kd[, keep_kd]
sf_df_kd <- read.delim("kneaddata/community_composition/normalized/scaling_factors.tsv")

fmt_p <- function(p) ifelse(p < 0.001, sprintf("p = %.2e", p), sprintf("p = %.4f", p))
to_factor <- function(df) mutate(df, type = factor(type, levels = type_levels))

total_per_sample <- colSums(counts_scaled)

total_df <- enframe(total_per_sample, name = "sample", value = "total_scaled") %>%
  left_join(meta, by = "sample") %>% filter(!is.na(type))

shannon_df <- diversity(t(counts_unscaled), index = "shannon") %>%
  enframe(name = "sample", value = "shannon") %>%
  left_join(meta, by = "sample") %>% filter(!is.na(type))

fuso_phylum_df <- colSums(counts_scaled[grepl("^Fusobacterium ",
  ↪ rownames(counts_scaled)), ]) %>%
  enframe(name = "sample", value = "fusobacteria") %>%
  left_join(meta, by = "sample") %>% filter(!is.na(type))

species_df <- data.frame(
  sample = colnames(counts_scaled),
  `Bacteroides fragilis` = as.numeric(counts_scaled["Bacteroides fragilis", ]),
  `Fusobacterium nucleatum` = as.numeric(counts_scaled["Fusobacterium nucleatum", ]),
  check.names = FALSE
) %>%

```

```

left_join(meta, by = "sample") %>% filter(!is.na(type)) %>%
mutate(bfrag_rel = `Bacteroides fragilis` / total_per_sample[sample],
       fnuc_rel = `Fusobacterium nucleatum` / total_per_sample[sample])

rel_long <- species_df %>%
  select(sample, type, bfrag_rel, fnuc_rel) %>%
  pivot_longer(c(bfrag_rel, fnuc_rel), names_to = "species", values_to = "rel") %>%
  mutate(species = recode(species, bfrag_rel = "B. fragilis", fnuc_rel = "F. nucleatum"),
         group = interaction(species, type, sep = "\n"),
         group = factor(group, levels = c(
           "B. fragilis\nCRC", "B. fragilis\nHealthy", "B. fragilis\nPaired",
           "F. nucleatum\nCRC", "F. nucleatum\nHealthy", "F. nucleatum\nPaired")))

# KneadData versions (86 samples)
total_per_sample_kd <- colSums(counts_scaled_kd)

total_df_kd <- enframe(total_per_sample_kd, name = "sample", value = "total_scaled") %>%
  left_join(meta, by = "sample") %>% filter(!is.na(type))

shannon_df_kd <- diversity(t(counts_unscaled_kd), index = "shannon") %>%
  enframe(name = "sample", value = "shannon") %>%
  left_join(meta, by = "sample") %>% filter(!is.na(type))

fuso_phylum_df_kd <- colSums(counts_scaled_kd[grepl("^Fusobacterium ",
  ↪ rownames(counts_scaled_kd)), ]) %>%
  enframe(name = "sample", value = "fusobacteria") %>%
  left_join(meta, by = "sample") %>% filter(!is.na(type))

species_df_kd <- data.frame(
  sample = colnames(counts_scaled_kd),
  `Bacteroides fragilis` = as.numeric(counts_scaled_kd["Bacteroides fragilis", ]),
  `Fusobacterium nucleatum` = as.numeric(counts_scaled_kd["Fusobacterium nucleatum", ]),
  check.names = FALSE
) %>%
  left_join(meta, by = "sample") %>% filter(!is.na(type)) %>%
  mutate(bfrag_rel = `Bacteroides fragilis` / total_per_sample_kd[sample],
       fnuc_rel = `Fusobacterium nucleatum` / total_per_sample_kd[sample])

rel_long_kd <- species_df_kd %>%
  select(sample, type, bfrag_rel, fnuc_rel) %>%
  pivot_longer(c(bfrag_rel, fnuc_rel), names_to = "species", values_to = "rel") %>%
  mutate(species = recode(species, bfrag_rel = "B. fragilis", fnuc_rel = "F. nucleatum"),
         group = interaction(species, type, sep = "\n"),
         group = factor(group, levels = c(
           "B. fragilis\nCRC", "B. fragilis\nHealthy", "B. fragilis\nPaired",
           "F. nucleatum\nCRC", "F. nucleatum\nHealthy", "F. nucleatum\nPaired")))

```

Figure S6 – Non-human read mapping per sample

```

fill_scale <- scale_fill_manual(
  name = "Mapping",
  values = c("B. fragilis" = "#A0522D", "F. nucleatum" = "#1B6CA8", "Unmapped" =
  ↪ "#BEBE9E")
)

```

```

sort_by_total <- function(df) {
  order <- df %>% group_by(sample) %>%
    summarise(t = sum(reads_M), .groups = "drop") %>% arrange(t) %>% pull(sample)
  mutate(df, sample = factor(sample, levels = order))
}

reads_long <- reads_df %>%
  left_join(meta %>% select(sample, type), by = "sample") %>% filter(!is.na(type)) %>%
  pivot_longer(c(bfrag, fnuc, unmapped), names_to = "mapping", values_to = "reads") %>%
  mutate(reads_M = reads / 1e6,
         mapping = factor(mapping, levels = c("bfrag", "fnuc", "unmapped"),
                          labels = c("B. fragilis", "F. nucleatum", "Unmapped")))

p_s6a <- reads_long %>% sort_by_total() %>%
  ggplot(aes(x = sample, y = reads_M, fill = mapping)) +
  geom_col(width = 1) + geom_hline(yintercept = 1, linetype = "dashed") +
  fill_scale + scale_y_continuous(expand = expansion(mult = c(0, 0.05))) +
  labs(x = "Sample", y = "Non-Human Reads (M)", title = "A All samples") +
  theme_classic() + theme(axis.text.x = element_blank(), axis.ticks.x = element_blank())

p_s6b <- reads_long %>% group_by(sample) %>% filter(sum(reads_M) > 1) %>% ungroup() %>%
  sort_by_total() %>%
  ggplot(aes(x = sample, y = reads_M, fill = mapping)) +
  geom_col() + fill_scale + scale_y_continuous(expand = expansion(mult = c(0, 0.05))) +
  labs(x = "Sample", y = "Non-Human Reads (M)", title = "B Samples with >1M reads") +
  theme_classic() + theme(axis.text.x = element_text(angle = 45, hjust = 1))

p_s6a + p_s6b + plot_layout(guides = "collect") +
  plot_annotation(title = "Figure S6 -- Non-human read mapping per sample")

```

Figure S6 -- Non-human read mapping per sample

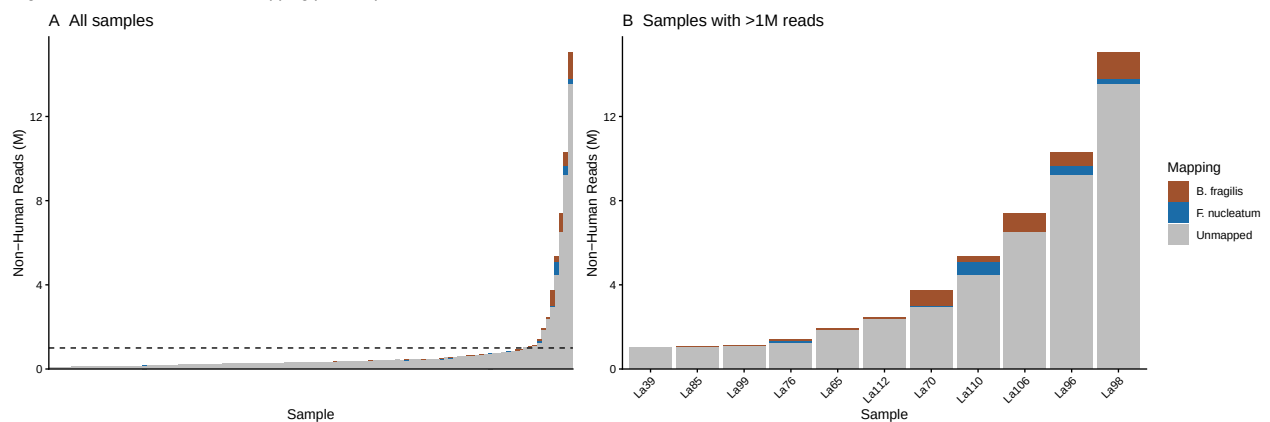


Table S2 – Top 20 expressed genes per organism (CRC samples, ≥ 1 M reads)

From Kvich et al. supplemental methods: reads normalized with DESeq2 variance-stabilizing transformation (VST); cutoff of 1M reads applied; expression averaged across passing samples; only protein-coding genes with a named product included.

```

gene_annot <- read.delim("no_kneaddata/alignment/gene_annotations.tsv") %>%
  filter(product != "")

```

```

crc_1M <- reads_df %>%
  left_join(meta %>% select(sample, type), by = "sample") %>%
  filter(type == "CRC", total_fragments >= 1e6) %>% pull(sample)

counts_mat <- counts_raw %>%
  column_to_rownames("Geneid") %>%
  select(all_of(sample_cols)) %>%
  select(any_of(crc_1M)) %>%
  mutate(across(everything(), as.integer))
coldata <- data.frame(row.names = colnames(counts_mat), sample = colnames(counts_mat))

run_vst <- function(gene_ids) {
  mat <- counts_mat[row.names(counts_mat) %in% gene_ids, ]
  mat <- mat[rowSums(mat) > 0, ]
  dds <- DESeqDataSetFromMatrix(mat, colData = coldata, design = ~ 1)
  rowMeans(assay(varianceStabilizingTransformation(dds, blind = TRUE)))
}

bfrag_avg <- run_vst(counts_raw %>% filter(Chr == BFRAG_CHR) %>% pull(Geneid))
fnuc_avg <- run_vst(counts_raw %>% filter(Chr == FNUC_CHR) %>% pull(Geneid))

make_top20 <- function(avg_expr, species_label) {
  data.frame(gene_id = names(avg_expr), avg_vst = avg_expr) %>%
    inner_join(gene_annot, by = "gene_id") %>%
    arrange(desc(avg_vst)) %>% slice_head(n = 20) %>%
    transmute(Species = species_label, Gene = gene, Product = product, Avg_VST =
      ↪ round(avg_vst, 2))
}

table_s2 <- bind_rows(make_top20(bfrag_avg, "B. fragilis"),
  make_top20(fnuc_avg, "F. nucleatum"))
knitr::kable(table_s2, caption = "Table S2 -- Top 20 expressed genes per organism in CRC
  ↪ samples with >=1M reads.")

```

Table 4: Table S2 – Top 20 expressed genes per organism in CRC samples with ≥ 1 M reads.

Species	Gene	Product	Avg_VST
B. fragilis	tuf	elongation factor Tu	14.10
B. fragilis	fusA	elongation factor G	13.45
B. fragilis	pckA	phosphoenolpyruvate carboxykinase (ATP)	12.29
B. fragilis	rplA	50S ribosomal protein L1	12.07
B. fragilis	rpoB	DNA-directed RNA polymerase subunit beta	12.02
B. fragilis	rpoC	DNA-directed RNA polymerase subunit beta'	12.02
B. fragilis	rpsA	30S ribosomal protein S1	11.95
B. fragilis	gap	type I glyceraldehyde-3-phosphate dehydrogenase	11.59
B. fragilis	rplB	50S ribosomal protein L2	11.47
B. fragilis	nifJ	pyruvate:ferredoxin (flavodoxin) oxidoreductase	11.45
B. fragilis	nusG	transcription termination/antitermination protein NusG	11.40
B. fragilis	groL	chaperonin GroEL	11.19
B. fragilis	rpsL	30S ribosomal protein S12	11.19
B. fragilis	rplK	50S ribosomal protein L11	10.98
B. fragilis		DNA-directed RNA polymerase subunit alpha	10.98

Species	Gene	Product	Avg_VST
B. fragilis	rpsG	30S ribosomal protein S7	10.97
B. fragilis		sigma-70 family RNA polymerase sigma factor	10.97
B. fragilis	rplL	50S ribosomal protein L7/L12	10.89
B. fragilis	rpsB	30S ribosomal protein S2	10.84
B. fragilis	rplS	50S ribosomal protein L19	10.82
F. nucleatum	tuf	elongation factor Tu	11.94
F. nucleatum	fusA	elongation factor G	10.69
F. nucleatum	gap	type I glyceraldehyde-3-phosphate dehydrogenase	10.14
F. nucleatum	nifJ	pyruvate:ferredoxin (flavodoxin) oxidoreductase	9.83
F. nucleatum	pflB	formate C-acetyltransferase	9.54
F. nucleatum	rpoC	DNA-directed RNA polymerase subunit beta'	9.24
F. nucleatum	rplL	50S ribosomal protein L7/L12	9.14
F. nucleatum	rpoB	DNA-directed RNA polymerase subunit beta	8.87
F. nucleatum	rplA	50S ribosomal protein L1	8.86
F. nucleatum		sodium-translocating pyrophosphatase	8.82
F. nucleatum	rpsB	30S ribosomal protein S2	8.74
F. nucleatum	dnaK	molecular chaperone DnaK	8.44
F. nucleatum	typA	translational GTPase TypA	8.22
F. nucleatum	rpsL	30S ribosomal protein S12	8.13
F. nucleatum	rplK	50S ribosomal protein L11	7.83
F. nucleatum	rplQ	50S ribosomal protein L17	7.82
F. nucleatum	groL	chaperonin GroEL	7.80
F. nucleatum	rpsI	30S ribosomal protein S9	7.80
F. nucleatum	ahpC	alkyl hydroperoxide reductase subunit C	7.79
F. nucleatum	ftsH	ATP-dependent zinc metalloprotease FtsH	7.67

Figure 2J - Virulence gene expression vs F. nucleatum abundance

fomA and radD are the two F. nucleatum virulence genes unambiguously annotated in GCF_016889925.1; featureCounts CDS counts are used in place of the authors' custom alignment.

```

FOMA_ID <- "C7Y58_RS03545"
RADD_ID <- "C7Y58_RS00155"

vir_counts <- counts_raw %>%
  filter(Geneid %in% c(FOMA_ID, RADD_ID)) %>%
  select(Geneid, all_of(sample_cols)) %>%
  pivot_longer(-Geneid, names_to = "sample", values_to = "raw_count") %>%
  left_join(sf_df %>% select(sample, scaling_factor), by = "sample") %>%
  mutate(scaled_count = raw_count * scaling_factor,
         gene = case_when(Geneid == FOMA_ID ~ "fomA", Geneid == RADD_ID ~ "radD"))

fnuc_row <- which(rownames(counts_scaled) == "Fusobacterium nucleatum")
fnuc_abund <- tibble(sample = colnames(counts_scaled),
                    fnuc_scaled = as.numeric(counts_scaled[fnuc_row, ]))

plot_df <- vir_counts %>%
  left_join(fnuc_abund, by = "sample") %>%
  left_join(meta %>% select(sample, type), by = "sample") %>%
  filter(!is.na(type), !is.na(fnuc_scaled)) %>%
  group_by(gene) %>%
  mutate(label = ifelse(fnuc_scaled > quantile(fnuc_scaled, 0.95), sample,
        NA_character_)) %>%

```

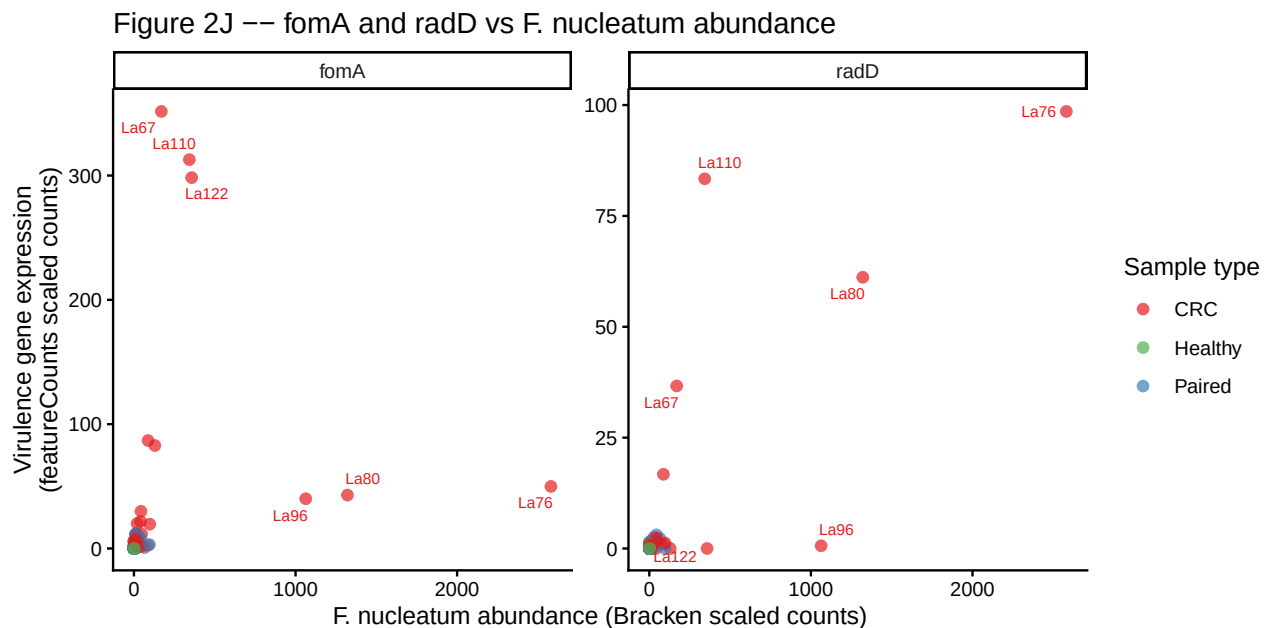
```

ungroup()

ggplot(plot_df, aes(x = fnuc_scaled, y = scaled_count, colour = type)) +
  geom_point(alpha = 0.7, size = 2) +
  geom_text_repel(aes(label = label), size = 2.5, max.overlaps = 20, show.legend = FALSE)
  ↪ +
  facet_wrap(~ gene, scales = "free_y") +
  scale_colour_manual(values = c("CRC" = "#E41A1C", "Healthy" = "#4DAF4A", "Paired" =
  ↪ "#377EB8"),
                      name = "Sample type") +
  labs(x = "F. nucleatum abundance (Bracken scaled counts)",
       y = "Virulence gene expression\n(featureCounts scaled counts)",
       title = "Figure 2J -- fomA and radD vs F. nucleatum abundance") +
  theme_classic()

```

Warning: Removed 224 rows containing missing values or values outside the scale range
 ## (`geom\_text\_repel()`).



- These three plots match the results reported by Kvich et al. for the gene expression analysis.

Figure 4 - Community composition by sample group

Two implementations are shown: the authors' original approach and a methodologically improved version. Key differences are summarised in the table below.

```

prep_paired <- function(df, value_col) {
  df %>%
    filter(type %in% c("CRC", "Paired"), patient_ID != 54) %>%
    group_by(patient_ID) %>% filter(n() == 2) %>% ungroup() %>%
    arrange(patient_ID, type) %>% select(patient_ID, type, all_of(value_col))
}

# Authors' p-values (one-sided, row-order pairing, Bonferroni across all comparisons per
  ↪ panel)
pA <- p.adjust(c(

```

```

total_df %>% filter(type %in% c("CRC","Healthy")) %>%
  wilcox_test(total_scaled ~ type, alternative = "greater") %>% pull(p),
prep_paired(total_df, "total_scaled") %>%
  wilcox_test(total_scaled ~ type, paired = TRUE, alternative = "greater") %>% pull(p)
))
pB <- p.adjust(c(
  shannon_df %>% filter(type %in% c("CRC","Healthy")) %>%
    t_test(shannon ~ type, alternative = "greater") %>% pull(p),
  shannon_df %>% filter(type %in% c("Paired","Healthy")) %>%
    t_test(shannon ~ type, alternative = "greater") %>% pull(p),
  prep_paired(shannon_df, "shannon") %>%
    t_test(shannon ~ type, paired = TRUE, alternative = "greater") %>% pull(p)
))
pC <- p.adjust(c(
  fuso_phylum_df %>% filter(type %in% c("CRC","Healthy")) %>%
    wilcox_test(fusobacteria ~ type) %>% pull(p),
  prep_paired(fuso_phylum_df, "fusobacteria") %>%
    wilcox_test(fusobacteria ~ type, paired = TRUE) %>% pull(p)
))
pD <- p.adjust(c(
  species_df %>% filter(type %in% c("CRC","Healthy")) %>%
    wilcox_test(fnuc_rel ~ type, alternative = "greater") %>% pull(p),
  prep_paired(species_df, "fnuc_rel") %>%
    wilcox_test(fnuc_rel ~ type, paired = TRUE, alternative = "greater") %>% pull(p),
  species_df %>% filter(type %in% c("CRC","Healthy")) %>%
    wilcox_test(bfrag_rel ~ type, alternative = "greater") %>% pull(p),
  prep_paired(species_df, "bfrag_rel") %>%
    wilcox_test(bfrag_rel ~ type, paired = TRUE, alternative = "greater") %>% pull(p)
))

# Improved p-values (two-sided, patient_ID join pairing, Bonferroni per panel)
paired_w2 <- function(df, col) {
  df %>% filter(type %in% c("CRC","Paired")) %>%
    select(patient_ID, type, all_of(col)) %>%
    pivot_wider(names_from = type, values_from = all_of(col)) %>% drop_na() %>%
    summarise(p = wilcox.test(CRC, Paired, paired = TRUE)$p.value) %>% pull(p)
}
paired_t2 <- function(df, col) {
  df %>% filter(type %in% c("CRC","Paired")) %>%
    select(patient_ID, type, all_of(col)) %>%
    pivot_wider(names_from = type, values_from = all_of(col)) %>% drop_na() %>%
    summarise(p = t.test(CRC, Paired, paired = TRUE)$p.value) %>% pull(p)
}
ind_w2 <- function(df, col) wilcox.test(df[[col]][df$type=="CRC"],
  ↪ df[[col]][df$type=="Healthy"])$p.value
ind_t2 <- function(df, col) t.test(df[[col]][df$type=="CRC"],
  ↪ df[[col]][df$type=="Healthy"])$p.value

qA <- p.adjust(c(ind_w2(total_df, "total_scaled"), paired_w2(total_df,
  ↪ "total_scaled")))
qB <- p.adjust(c(ind_t2(shannon_df, "shannon"), paired_t2(shannon_df,
  ↪ "shannon")))
qC <- p.adjust(c(ind_w2(fuso_phylum_df, "fusobacteria"), paired_w2(fuso_phylum_df,
  ↪ "fusobacteria")))

```

```
## Warning in wilcox.test.default(df[[col]][df$type == "CRC"], df[[col]][df$type == "CRC"]) : cannot compute exact p-value with ties
```

```
## Warning: There was 1 warning in `summarise()`.
## i In argument: `p = wilcox.test(CRC, Paired, paired = TRUE)$p.value`.
## Caused by warning in `wilcox.test.default()`:
## ! cannot compute exact p-value with zeroes
```

```
qD_b <- p.adjust(c(ind_w2(species_df, "bfrag_rel"), paired_w2(species_df,
  ↪ "bfrag_rel")))
```

```
## Warning in wilcox.test.default(df[[col]][df$type == "CRC"], df[[col]][df$type == "CRC"]) : cannot compute exact p-value with ties
## Warning in wilcox.test.default(df[[col]][df$type == "CRC"], df[[col]][df$type == "CRC"]) : There was 1 warning in `summarise()`.
## i In argument: `p = wilcox.test(CRC, Paired, paired = TRUE)$p.value`.
## Caused by warning in `wilcox.test.default()`:
## ! cannot compute exact p-value with zeroes
```

```
qD_f <- p.adjust(c(ind_w2(species_df, "fnuc_rel"), paired_w2(species_df,
  ↪ "fnuc_rel")))
```

```
## Warning in wilcox.test.default(df[[col]][df$type == "CRC"], df[[col]][df$type == "CRC"]) : cannot compute exact p-value with ties
## Warning in wilcox.test.default(df[[col]][df$type == "CRC"], df[[col]][df$type == "CRC"]) : There was 1 warning in `summarise()`.
## i In argument: `p = wilcox.test(CRC, Paired, paired = TRUE)$p.value`.
## Caused by warning in `wilcox.test.default()`:
## ! cannot compute exact p-value with zeroes
```

```
# KneadData p-values (improved methodology)
qA_kd <- p.adjust(c(ind_w2(total_df_kd, "total_scaled"), paired_w2(total_df_kd,
  ↪ "total_scaled"))
qB_kd <- p.adjust(c(ind_t2(shannon_df_kd, "shannon"), paired_t2(shannon_df_kd,
  ↪ "shannon"))
qC_kd <- p.adjust(c(ind_w2(fuso_phylum_df_kd, "fusobacteria"),
  ↪ paired_w2(fuso_phylum_df_kd, "fusobacteria")))
```

```
## Warning in wilcox.test.default(df[[col]][df$type == "CRC"], df[[col]][df$type == "CRC"]) : cannot compute exact p-value with ties
## Warning in wilcox.test.default(df[[col]][df$type == "CRC"], df[[col]][df$type == "CRC"]) : There was 1 warning in `summarise()`.
## i In argument: `p = wilcox.test(CRC, Paired, paired = TRUE)$p.value`.
## Caused by warning in `wilcox.test.default()`:
## ! cannot compute exact p-value with zeroes
```

```
qD_b_kd <- p.adjust(c(ind_w2(species_df_kd, "bfrag_rel"), paired_w2(species_df_kd,
  ↪ "bfrag_rel")))
```

```
## Warning in wilcox.test.default(df[[col]][df$type == "CRC"], df[[col]][df$type == "CRC"]) : cannot compute exact p-value with ties
```

```
qD_f_kd <- p.adjust(c(ind_w2(species_df_kd, "fnuc_rel"), paired_w2(species_df_kd,
  ↪ "fnuc_rel")))
```

```
## Warning in wilcox.test.default(df[[col]][df$type == "CRC"], df[[col]][df$type == "CRC"]) : cannot compute exact p-value with ties
## Warning in wilcox.test.default(df[[col]][df$type == "CRC"], df[[col]][df$type == "CRC"]) : There was 1 warning in `summarise()`.
## i In argument: `p = wilcox.test(CRC, Paired, paired = TRUE)$p.value`.
## Caused by warning in `wilcox.test.default()`:
## ! cannot compute exact p-value with zeroes
```

```
# Convert to factor after all stat tests
total_df <- to_factor(total_df); total_df_kd <- to_factor(total_df_kd)
```

```

shannon_df      <- to_factor(shannon_df);      shannon_df_kd      <-
  ↪ to_factor(shannon_df_kd)
fuso_phylum_df <- to_factor(fuso_phylum_df); fuso_phylum_df_kd <-
  ↪ to_factor(fuso_phylum_df_kd)
species_df      <- to_factor(species_df);      species_df_kd      <-
  ↪ to_factor(species_df_kd)
rel_long        <- mutate(rel_long,      type = factor(type, levels = type_levels))
rel_long_kd     <- mutate(rel_long_kd, type = factor(type, levels = type_levels))

```

Reproduction of the authors' Figure 4 (one-sided tests, geom_sina, row-order pairing as in the original):

```

pa_A <- ggplot(total_df, aes(x = type, y = total_scaled / 1e6)) +
  geom_boxplot(width = 0.4, color = "grey40", outlier.shape = NA) +
  geom_sina(color = "grey30", size = 1.5, maxwidth = 0.5, alpha = 0.8) +
  geom_signif(comparisons = list(c("CRC", "Healthy"), c("CRC", "Paired")),
    annotations = fmt_p(pA), step_increase = 0.12, tip_length = 0.02, textsize
    ↪ = 3) +
  ylim(0, max(total_df$total_scaled/1e6) * 1.3) +
  labs(x = NULL, y = "Scaled Counts (x10^6)",
    title = "A Scaled counts assigned to\nBacteria superkingdom") + theme_bw()
pa_B <- ggplot(shannon_df, aes(x = type, y = shannon)) +
  geom_boxplot(outlier.shape = NA) + geom_jitter(position = position_jitter(0.1)) +
  geom_signif(comparisons = list(c("CRC", "Healthy"), c("CRC", "Paired"),
    ↪ c("Healthy", "Paired")),
    annotations = fmt_p(pB), step_increase = 0.1, tip_length = 0.02, textsize =
    ↪ 3) +
  ylim(0, max(shannon_df$shannon) * 1.35) +
  labs(x = NULL, y = "Shannon Index", title = "B Alpha Diversity") + theme_bw()
pa_C <- ggplot(fuso_phylum_df, aes(x = type, y = fusobacteria)) +
  geom_jitter(position = position_jitter(0.1)) +
  geom_signif(comparisons = list(c("CRC", "Healthy"), c("CRC", "Paired")),
    annotations = fmt_p(pC), step_increase = 0.12, tip_length = 0.02, textsize
    ↪ = 3) +
  ylim(0, max(fuso_phylum_df$fusobacteria) * 1.3) +
  labs(x = NULL, y = "Scaled Counts", title = "C Fusobacteria Counts") + theme_bw()
pa_D <- ggplot(rel_long, aes(x = group, y = rel)) +
  geom_boxplot(outlier.shape = NA) + geom_jitter(width = 0.2, size = 0.8) +
  geom_signif(comparisons = list(c("B. fragilis\nCRC", "B. fragilis\nHealthy"),
    c("F. nucleatum\nCRC", "F. nucleatum\nHealthy")),
    annotations = fmt_p(pD[c(3,1)]),
    step_increase = 0.1, tip_length = 0.02, textsize = 3) +
  labs(x = NULL, y = "Relative abundance",
    title = "D B. fragilis and F. nucleatum\nrelative abundance") +
  theme_bw() + theme(axis.text.x = element_text(size = 8))
(pa_A | pa_B) / (pa_C | pa_D) +
  plot_annotation(title = "Figure 4 -- Reproduction of Kvich et al. (raw reads, n=119)")

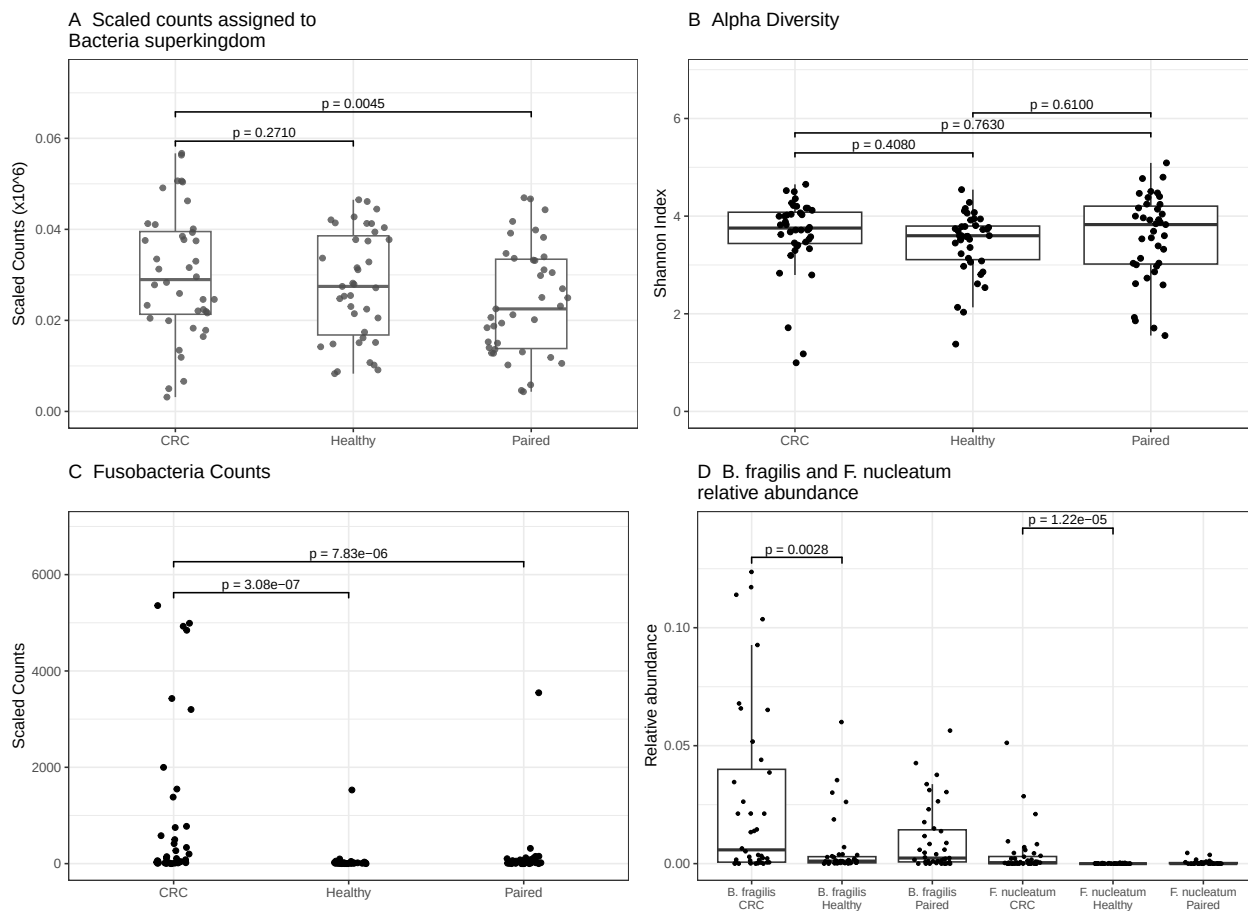
```

```

## Warning: Removed 17 rows containing missing values or values outside the scale range
## (`geom_point()`).

```

Figure 4 -- Reproduction of Kvich et al. (raw reads, n=119)



- I did not agree with the author's choice to do a one sided test (CRC > others) for panels A, B, and D. I chose to carry forward with a two sided test throughout. Since most of the patients are paired (CRC vs paired), I chose to do paired testing. Key methodological differences between the reproduction and improved implementation below:

Figure 4 repeated on KneadData-cleaned reads (86 samples) using the improved methodology:

```
make_fig4_panels <- function(total, shannon, fuso, rel, qA, qB, qC, qD_b, qD_f, subtitle)
{
  pA <- ggplot(total, aes(x = type, y = total_scaled / 1e6)) +
    geom_boxplot(outlier.shape = NA) + geom_jitter(width = 0.2, size = 0.8) +
    geom_signif(comparisons = list(c("CRC", "Healthy"), c("CRC", "Paired")),
      annotations = fmt_p(qA), step_increase = 0.12, tip_length = 0.02,
      <-> textsizesize = 3) +
    ylim(0, max(total$total_scaled/1e6) * 1.3) +
    labs(x = NULL, y = "Scaled Counts (x10^6)",
      title = "A Bacterial scaled counts", subtitle = subtitle) + theme_classic()
  pB <- ggplot(shannon, aes(x = type, y = shannon)) +
    geom_boxplot(outlier.shape = NA) + geom_jitter(width = 0.2, size = 0.8) +
    geom_signif(comparisons = list(c("CRC", "Healthy"), c("CRC", "Paired")),
      annotations = fmt_p(qB), step_increase = 0.1, tip_length = 0.02, textsizesize
      <-> = 3) +
    ylim(0, max(shannon$shannon) * 1.3) +
    labs(x = NULL, y = "Shannon Index", title = "B Alpha Diversity", subtitle =
      <-> subtitle) + theme_classic()
}
```

```

pC <- ggplot(fuso, aes(x = type, y = fusobacteria)) +
  geom_boxplot(outlier.shape = NA) + geom_jitter(width = 0.2, size = 0.8) +
  geom_signif(comparisons = list(c("CRC", "Healthy"), c("CRC", "Paired")),
    annotations = fmt_p(qC), step_increase = 0.12, tip_length = 0.02,
    ↪     textsize = 3) +
  ylim(0, max(fuso$fusobacteria) * 1.3) +
  labs(x = NULL, y = "Scaled Counts", title = "C Fusobacteria Counts", subtitle =
    ↪     subtitle) + theme_classic()
pD <- ggplot(rel, aes(x = group, y = rel)) +
  geom_boxplot(outlier.shape = NA) + geom_jitter(width = 0.2, size = 0.8) +
  geom_signif(comparisons = list(c("B. fragilis\nCRC", "B. fragilis\nHealthy"),
    c("F. nucleatum\nCRC", "F. nucleatum\nHealthy")),
    annotations = fmt_p(c(qD_b[1], qD_f[1])),
    step_increase = 0.1, tip_length = 0.02, textsize = 3) +
  labs(x = NULL, y = "Relative abundance",
    title = "D B. fragilis / F. nucleatum relative abundance", subtitle = subtitle)
    ↪     +
  theme_classic() + theme(axis.text.x = element_text(size = 8))
(pA | pB) / (pC | pD)
}

make_fig4_panels(total_df_kd, shannon_df_kd, fuso_phylum_df_kd, rel_long_kd,
  qA_kd, qB_kd, qC_kd, qD_b_kd, qD_f_kd, "KneadData-cleaned (n=86)") +
  plot_annotation(title = "Figure 4 -- KneadData-cleaned reads")

```

```

## Warning: Removed 10 rows containing missing values or values outside the scale range
## (`geom_point()`).

```

Figure 4 -- KneadData-cleaned reads

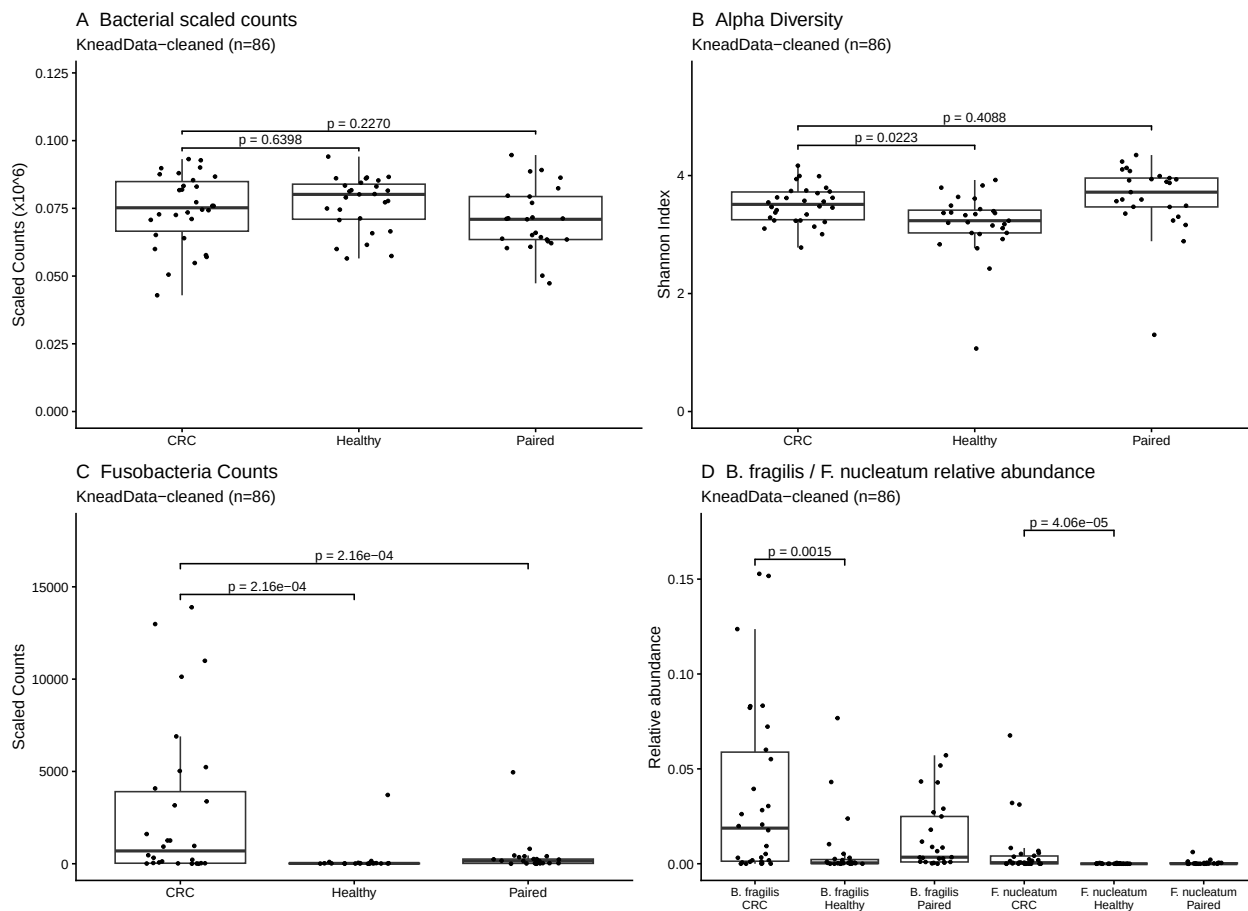
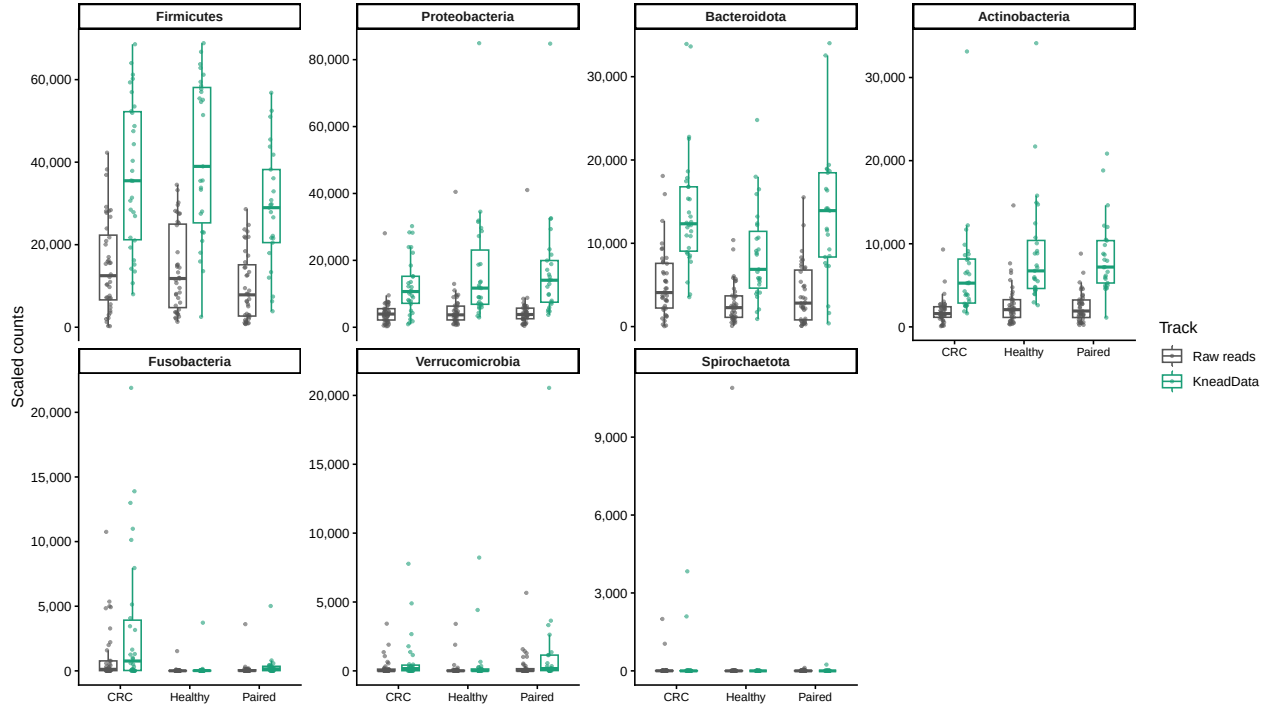


Figure S4 – Phylum-level bacterial abundance

Phylum names from the 2026 Kraken2 database are harmonized to traditional names used in the paper (e.g. Bacillota → Firmicutes). Campylobacterota and related phyla are merged into Proteobacteria to match the older taxonomy used by the original authors. Species-level Bracken `counts_scaled` are summed by phylum using a `taxonomy_id` → phylum map built from the Kraken2 report tree (replicating the authors' taxize-based approach without API calls).

```
phylum_df %>%
  filter(phylum != "Other") %>%
  ggplot(aes(x = type, y = scaled, colour = track)) +
  geom_boxplot(outlier.shape = NA, position = position_dodge(0.7), width = 0.5) +
  geom_jitter(position = position_jitterdodge(jitter.width = 0.15, dodge.width = 0.7),
    size = 0.7, alpha = 0.6) +
  facet_wrap(~phylum, scales = "free_y", nrow = 2) +
  scale_colour_manual(values = c("Raw reads" = "#555555", "KneadData" = "#1B9E77"),
    name = "Track") +
  scale_y_continuous(labels = scales::comma) +
  labs(x = NULL, y = "Scaled counts",
    title = "Figure S4B -- Scaled counts for top phyla by sample group") +
  theme_classic() + theme(strip.text = element_text(face = "bold"),
    axis.text.x = element_text(size = 8))
```

Figure S4B -- Scaled counts for top phyla by sample group



Part 2: Further pre-processing of unmapped reads with KneadData

Although the Kvich et al. unmapped reads had been adapter-trimmed and rRNA-depleted upstream, I ran the samples through another round of pre-processing to ensure no residual human sequences were carrying forward. I used the bioBakery's KneadData v0.12.4 (<https://huttenhower.sph.harvard.edu/kneaddata/>), which combines Trimmomatic quality trimming with Bowtie2-based human read decontamination against T2T-CHM13v2.0 (hg39), and evaluate its effect on read quality, sample depth, and community composition relative to the Part 1 baseline.

Scripts: `scripts/kneaddata/01_kneaddata.sh` through `07_normalize_bracken.py` Local output: `kneaddata/`

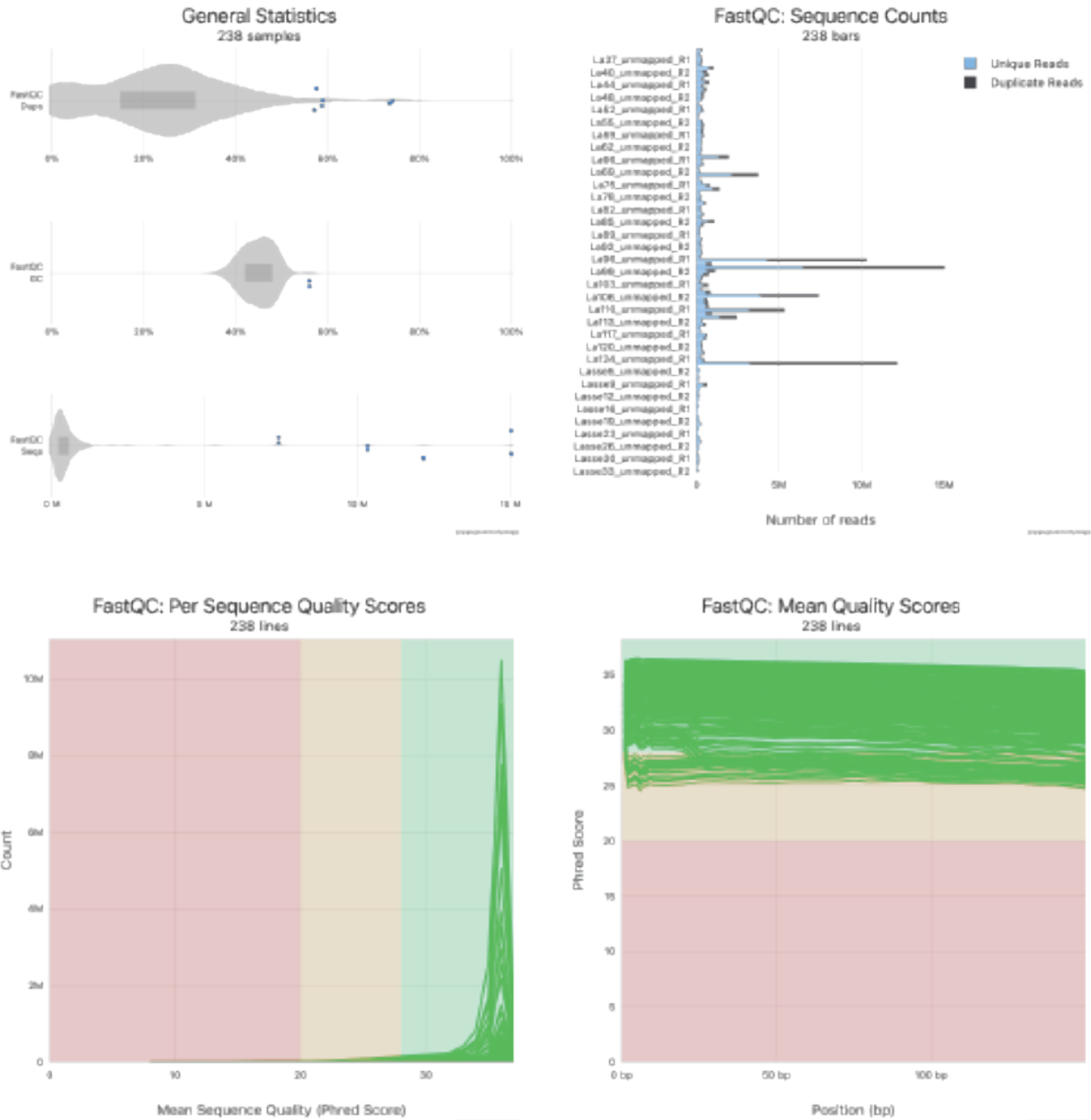
2.1 Pre-KneadData quality assessment (FastQC + MultiQC)

FastQC was run on all 238 raw unmapped FASTQ files to establish a quality baseline before KneadData.

```
fastqc input/*.fq --outdir output/qc/fastqc_pre --threads 16
```

```
multiqc output/qc/fastqc_pre \
  --outdir output/qc/multiqc_pre --filename multiqc_report
```

Reports: `kneaddata/qc/multiqc_pre/multiqc_report.html`.



Read counts vary widely across biopsy samples, which is consistent with the differences in microbial biomass seen throughout the dataset. Per-read quality scores were generally high across all samples.

2.2 Human read removal and quality trimming (script: 01_kneaddata.sh)

KneadData runs Trimmomatic quality trimming and Bowtie2 human decontamination in a single pass. `--decontaminate-pairs strict` (the default) removes a read pair if either mate maps to the human reference. `--bypass-trf` skips tandem repeat filtering, which is not relevant for RNA-seq. Intermediate files were removed after each sample.

SLURM array: 119 tasks, 8 CPUs, 32 GB RAM, 12 h.

```
kneaddata \
  --input1 input/{sample}_unmapped_R1.fq \
  --input2 input/{sample}_unmapped_R2.fq \
```

```

--reference-db dbs/human_genome/hg_39 \
--output kneaddata_output/{sample}/ \
--threads 8 \
--trimmomatic /home/alicesh/miniconda3/envs/bfrag4/share/trimmomatic-0.40-0 \
--trimmomatic-options "SLIDINGWINDOW:4:20 MINLEN:50 LEADING:3 TRAILING:3" \
--bypass-trf --remove-intermediate-output \
--log kneaddata_output/{sample}/{sample}.log

```

Trimmomatic parameters: SLIDINGWINDOW:4:20 trims when a 4-base window quality drops below Q20; MINLEN:50 discards reads shorter than 50 bp; LEADING:3 TRAILING:3 removes bases with quality < 3 from read ends.

Read counts at each stage compiled with:

```

kneaddata_read_count_table \
--input kneaddata_output/ --output kneaddata/kneaddata_read_counts.tsv

```

Output: kneaddata/kneaddata_read_counts.tsv.

2.3 Post-KneadData quality assessment (scripts: 02_fastqc.sh, 03_multiqc.sh)

FastQC was re-run on the cleaned paired output files. MultiQC reports were generated for post-KneadData reads alone and as a combined pre/post comparison. MultiQC was run from the base conda environment (/home/alicesh/miniconda3/bin/multiqc) – the bfrag4 environment has a broken metadata issue. SLURM: 16 CPUs, 32 GB RAM, 12 h (FastQC); 4 CPUs, 16 GB RAM, 1 h (MultiQC).

```

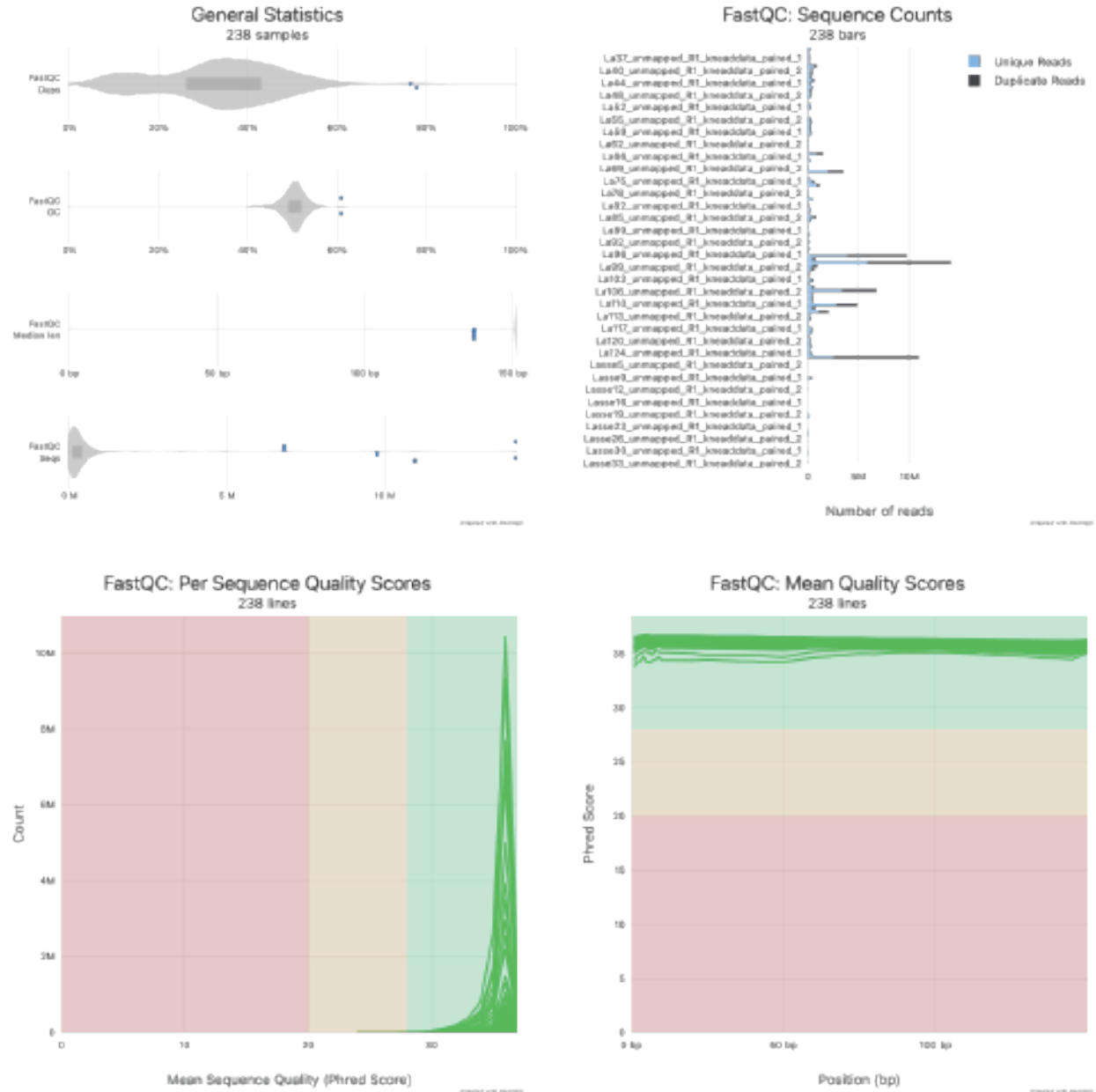
fastqc kneaddata_output/**/*.kneaddata_paired_*.fastq \
--outdir output/qc/fastqc_post --threads 16

multiqc output/qc/fastqc_post \
--outdir output/qc/multiqc_post --filename multiqc_report

multiqc output/qc/fastqc_pre output/qc/fastqc_post \
--outdir output/qc/multiqc_comparison --filename multiqc_comparison

```

Reports: kneaddata/qc/multiqc_post/multiqc_report.html and kneaddata/qc/multiqc_comparison/multiqc_comparison.html



Total read counts are reduced across samples after filtering, while read quality remains high. The large variation in retained depth between samples remains, which is expected for low-biomass biopsy-derived microbial reads.

2.4 Sample exclusions

33 samples were excluded due to insufficient read depth or high QC failure rates after KneadData:

- **Lasse samples:** Lasse4–Lasse33 except Lasse9, Lasse20, and Lasse25 (27 excluded)
- **La samples:** La50, La54, La68, La73, La77, La89 (6 excluded)

The 86 retained samples are listed in `samples_filtered.txt`.

2.5 Taxonomic profiling – Kraken2 (script: 04_kraken2.sh)

Same database and confidence threshold as Part 1. SLURM array: 86 tasks, 8 CPUs, 100 GB RAM, 24 h.

```
kraken2 \
  --db /scratch/st-ctropini-1/ahong/kraken2_standard_db \
  --threads 8 --confidence 0.05 --paired \
  --report output/kraken2/{sample}.report --output /dev/null \
  kneaddata_output/{sample}/{sample}_unmapped_R1_kneaddata_paired_1.fastq \
  kneaddata_output/{sample}/{sample}_unmapped_R1_kneaddata_paired_2.fastq
```

Output: kneaddata/community_composition/kraken2/{sample}.report (86 files).

2.6 Abundance estimation – Bracken (script: 05_bracken.sh)

SLURM array: 86 tasks, 4 CPUs, 16 GB RAM, 4 h. Same parameters as Part 1.

```
bracken -d /scratch/st-ctropini-1/ahong/kraken2_standard_db \
  -i output/kraken2/{sample}.report \
  -o output/bracken/{sample}.bracken \
  -w output/bracken/{sample}_bracken.report \
  -r 150 -l S
```

Output: kneaddata/community_composition/bracken/ – 172 files (2 per sample).

2.7 Normalize Bracken counts (script: 07_normalize_bracken.py)

Same Kvich et al. normalization as Part 1; read counts from post-KneadData paired R1 FASTQ. Minimum-depth sample post-KneadData: La53 (101,423 reads).

```
conda activate kraken_env
python scripts/kneaddata/07_normalize_bracken.py
```

Output in kneaddata/community_composition/normalized/: counts_unscaled.tsv, counts_scaled.tsv, counts_filtered.tsv, scaling_factors.tsv.

2.8 Results: KneadData read quality and sample retention

Community composition comparisons between raw and KneadData-cleaned reads are in section 1.9. This section covers read-level QC outcomes: read fate, depth distribution, excluded samples, and Kraken2 classification rates.

```
# [R analysis -- to be added]
# - Plot read fate across KneadData stages per sample:
#   stacked bar of retained / removed by QC trim / removed by human decontamination
#   (from kneaddata/kneaddata_read_counts.tsv)
# - Plot post-KneadData read depth distribution with exclusion threshold marked
# - Table of excluded samples: sample ID, raw depth, KneadData depth, clinical group
# - Verify exclusions are not systematically biased by clinical group
# - Compare Kraken2 classification rates between raw and KneadData tracks
#   (for the 86 overlapping samples): % classified, % unclassified
# Note: community composition comparisons (Figure 4, Figure S4) are shown
# side-by-side with the raw reads results in section 1.9 above.
```

Part 3: Functional Profiling with HUMANN3

HUMANN3 functional profiling was applied to the KneadData-cleaned reads (84 samples; La98 and La125 excluded). This analysis, which was not performed in the original study, characterizes community-wide and species-resolved metabolic activity across CRC, paired normal, and healthy tissue.

HUMANn3 v3.6 was used (not v4, which was still in alpha at the time of analysis). HUMANn3 accepts a single-end input file and a taxonomic pre-screen in MetaPhlAn3 format, requiring two preparatory steps per sample: converting the Bracken report to MetaPhlAn3 format, and concatenating R1 + R2 into a single FASTQ.

Note: La98 and La125 were excluded: both are the highest read-depth outliers (La98: ~15M reads, La125: ~12.2M reads vs. median ~0.64M) and did not finish within the 24 h wall time.

3.1 Convert Bracken report to MetaPhlAn3 format (script: 06_bracken_to_mpa.py)

HUMANn3's `--taxonomic-profile` argument accepts a MetaPhlAn3-format profile to bypass its internal MetaPhlAn step. The KrakenTools `kreport2mpa.py` script was not used due to compatibility issues; a custom script was written. It reads the Bracken-corrected `_bracken.report` and emits only the 7 standard NCBI ranks (Domain to Species) using MetaPhlAn3 prefix notation (`k__p__c__o__f__g__s__`). Species absent from ChocoPhlAn fall back to the UniRef90 translated search; they still contribute to pathway/gene-family tables but without nucleotide-level hits.

This step runs within the HUMANn3 SLURM script but can also be run independently:

```
python scripts/kneaddata/06_bracken_to_mpa.py \
    kneaddata/community_composition/bracken/{sample}_bracken.report \
    kneaddata/functional_profiling/humann3_mpa/{sample}.mpa
```

Output: `kneaddata/functional_profiling/humann3_mpa/` (86 .mpa files).

3.2 Run HUMANn3 (script: 08_humann3.sh for each track)

Three steps per sample: (1) convert Bracken report to MetaPhlAn3 format (section 3.1); (2) concatenate R1 + R2 – HUMANn3 treats reads independently so concatenation loses no information, and the concatenated file is named `{sample}_cat.fq`, hence all HUMANn3 outputs contain `_cat_`; (3) run HUMANn3.

```
cat {R1} {R2} > output/humann3_reads/{sample}_cat.fq

humann \
    --input output/humann3_reads/{sample}_cat.fq \
    --taxonomic-profile output/humann3_mpa/{sample}.mpa \
    --output output/humann3/{sample} \
    --threads 16 \
    --nucleotide-database
    ↪ /home/alicesh/project/ahong/databases/humann_databases/chocophlan \
    --protein-database /home/alicesh/project/ahong/databases/humann_databases/uniref \
    --memory-use maximum

rm -f output/humann3_reads/{sample}_cat.fq
```

SLURM: 16 CPUs, 64 GB RAM, 24 h per task; `--array=0-85` (86 samples; La98 and La125 excluded).

Output per sample: `{sample}_cat_genefamilies.tsv` (UniRef90 gene families, RPK), `{sample}_cat_pathabundance.tsv` (MetaCyc pathway abundances, RPK), `{sample}_cat_pathcoverage.tsv` (MetaCyc pathway coverage, 0–1).

3.3 HUMANn3 post-processing

Once all samples were complete, per-sample tables are merged, normalized to copies per million (CPM), and split into stratified and unstratified components.

```
conda activate humann3
```

```

humann_join_tables -i kneaddata/functional_profiling/humann3 --file_name genefamilies \
  -o kneaddata/functional_profiling/humann3_merged/all_genefamilies.tsv

humann_join_tables -i kneaddata/functional_profiling/humann3 --file_name pathabundance \
  -o kneaddata/functional_profiling/humann3_merged/all_pathabundance.tsv

humann_renorm_table \
  --input kneaddata/functional_profiling/humann3_merged/all_genefamilies.tsv \
  --units cpm \
  --output kneaddata/functional_profiling/humann3_merged/all_genefamilies_cpm.tsv

humann_split_stratified_table \
  --input kneaddata/functional_profiling/humann3_merged/all_pathabundance.tsv \
  --output kneaddata/functional_profiling/humann3_merged/

```

3.4 Results: Community-wide functional profiling

```

KD_MERGED <- "kneaddata/functional_profiling/humann3_merged"

# Unstratified pathway abundance (CPM) - 390 pathways x 84 samples
path_abund <- read.delim(file.path(KD_MERGED,
  ↪ "all_pathabundance_unstratified.tsv"), check.names = FALSE) %>%
  rename_with(~"pathway", 1) %>%
  filter(!pathway %in% c("UNMAPPED", "UNINTEGRATED"))

# Pathway coverage (0-1) - same dimensions, used to filter low-confidence pathways
path_cov <- read.delim(file.path(KD_MERGED, "all_pathcoverage.tsv"), check.names = FALSE)
  ↪ %>%
  rename_with(~"pathway", 1) %>%
  filter(!pathway %in% c("UNMAPPED", "UNINTEGRATED"))

# Stratified pathway abundance - pathway/species rows
path_strat <- read.delim(file.path(KD_MERGED,
  ↪ "all_pathabundance_stratified.tsv"), check.names = FALSE) %>%
  rename_with(~"pathway_species", 1) %>%
  filter(!grepl("^UNMAPPED|UNINTEGRATED", pathway_species))

# Clean up sample column names: strip "_cat_Abundance" suffix, remap Lasse -> La
clean_humann_cols <- function(df) {
  colnames(df) <- colnames(df) %>%
    str_remove("_cat_Abundance$") %>%
    str_remove("_cat_Coverage$") %>%
    str_replace("^Lasse(\\d+)$", "La\\1")
  df
}

path_abund <- clean_humann_cols(path_abund)
path_cov <- clean_humann_cols(path_cov)
path_strat <- clean_humann_cols(path_strat)

# Sample columns present in both HUMANN3 output and metadata
h3_samples <- intersect(colnames(path_abund)[-1], meta$sample)
meta_h3 <- meta %>% filter(sample %in% h3_samples) %>%

```

```

mutate(type = factor(type, levels = type_levels))

# Coverage-filtered pathway matrix: keep pathways with mean coverage >= 0.1
cov_mat <- path_cov %>% column_to_rownames("pathway") %>% select(all_of(h3_samples))
path_mat <- path_abund %>% column_to_rownames("pathway") %>% select(all_of(h3_samples))
keep_paths <- rowMeans(cov_mat >= 0.1, na.rm = TRUE) >= 0.1
path_mat_filt <- path_mat[keep_paths, ]
path_mat_filt[is.na(path_mat_filt)] <- 0

# Long format for stats/plots
path_long <- path_mat_filt %>%
  rownames_to_column("pathway") %>%
  pivot_longer(-pathway, names_to = "sample", values_to = "cpm") %>%
  left_join(meta_h3, by = "sample")

path_mat_num <- path_mat_filt %>% as.matrix()

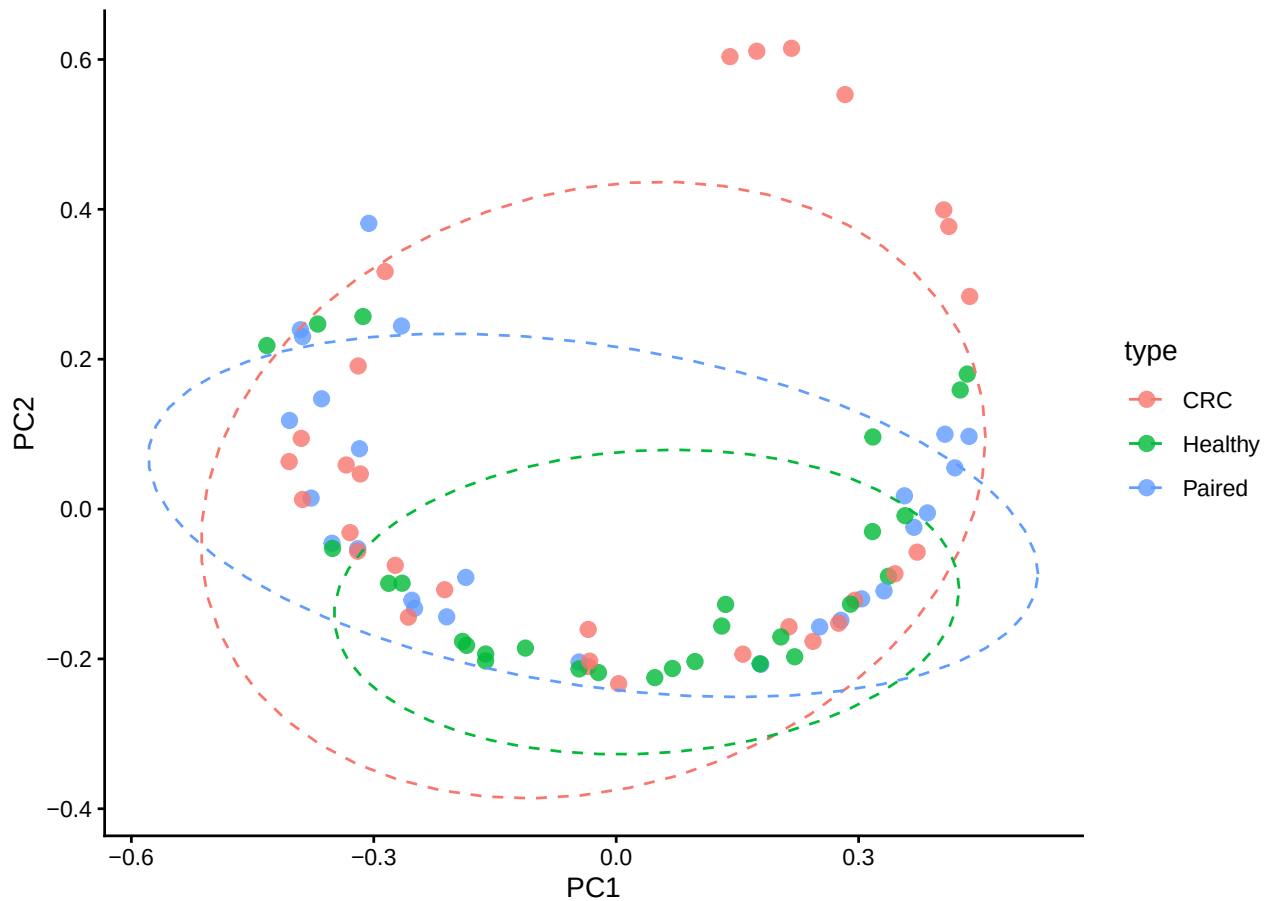
mode(path_mat_num) <- "numeric"
path_mat_num[is.na(path_mat_num)] <- 0

bc_dist <- vegdist(t(path_mat_num), method = "bray")

pcoa <- cmdscale(bc_dist, k = 2, eig = TRUE)
pct_var <- round(pcoa$eig[1:2] / sum(pcoa$eig[pcoa$eig > 0]) * 100, 1)

# plotting pcoa2
data.frame(sample = rownames(pcoa$points), PC1 = pcoa$points[,1], PC2 = pcoa$points[,2])
↪ %>%
  left_join(meta_h3, by = "sample") %>%
  ggplot(aes(PC1, PC2, colour = type)) +
  geom_point(size = 2.5, alpha = 0.8) +
  stat_ellipse(level = 0.75, linetype = "dashed") +
  theme_classic()

```



- The arc shape shows that the primary axis of functional variation follows a continuous gradient rather than discrete disease-associated clustering (between CRC and healthy or paired), reflecting the differences in microbial biomass we already observed and pathway detection sensitivity across low-biomass biopsy samples.

```
# Kruskal-Wallis across groups per pathway, BH correction
# kw_res <- path_long %>%
#   group_by(pathway) %>%
#   summarise(p = kruskal.test(cpm ~ type)$p.value, .groups = "drop") %>%
#   mutate(p_adj = p.adjust(p, method = "BH")) %>%
#   arrange(p_adj)
kw_res <- path_long %>%
  filter(!is.na(pathway), pathway != "NA", !is.na(type), is.finite(cpm)) %>%
  group_by(pathway) %>%
  filter(sum(cpm > 0) >= 5, n_distinct(type[cpm > 0]) >= 2) %>%
  summarise(
    p = kruskal.test(cpm ~ droplevels(type))$p.value,
    .groups = "drop"
  ) %>%
  mutate(p_adj = p.adjust(p, method = "BH")) %>%
  arrange(p_adj)

kw_res %>%
  slice_head(n = 10) %>%
  mutate(pathway = str_remove(pathway, "[^:]+: ") %>% str_trunc(60),
         p       = signif(p, 3),
         p_adj   = signif(p_adj, 3)) %>%
```

```
dplyr::rename(Pathway = pathway, `p (KW)` = p, `p adj (BH)` = p_adj) %>%
knitr::kable(caption = "Top 10 pathways by Kruskal-Wallis test across sample groups
↪ (BH-adjusted).")
```

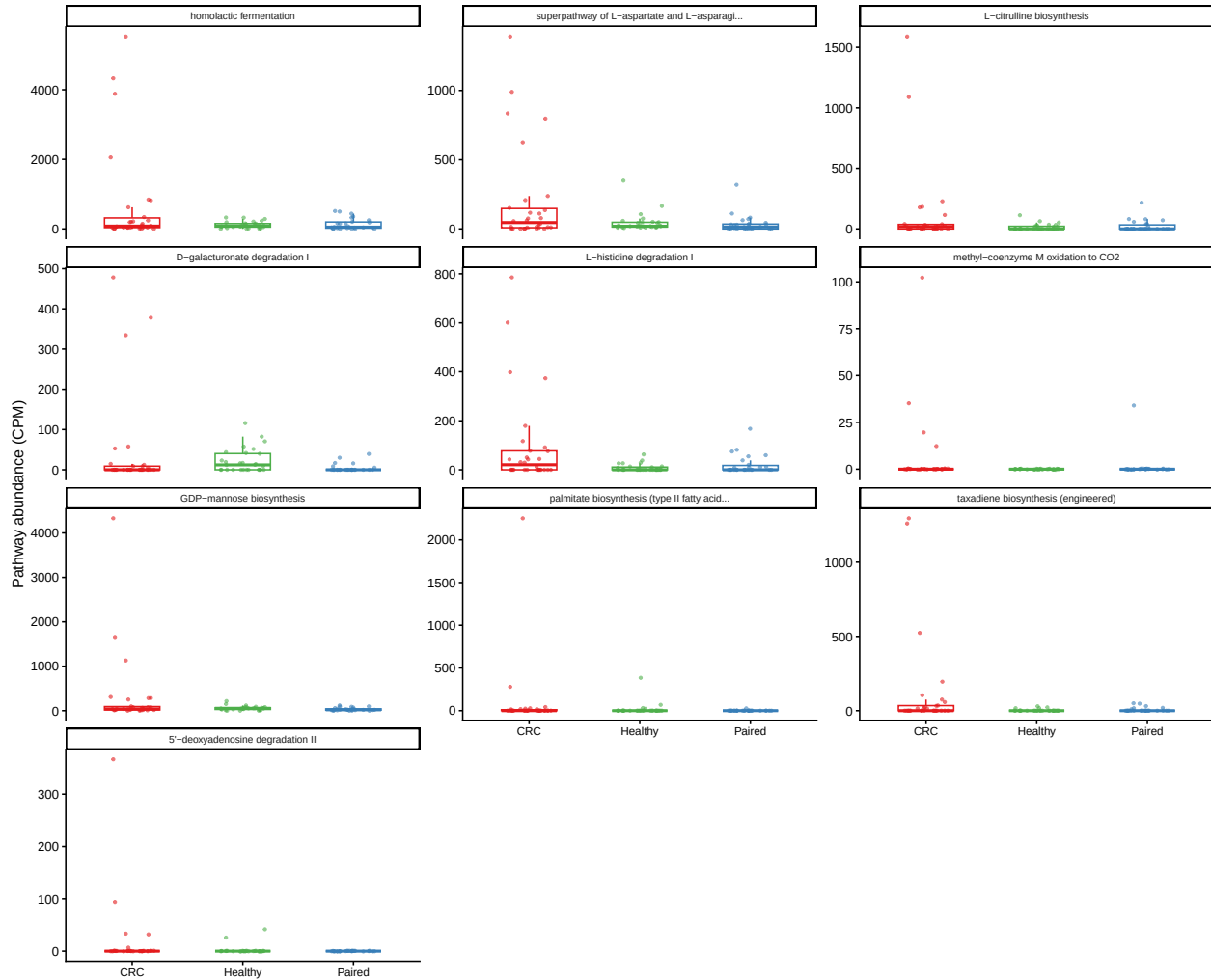
Table 5: Top 10 pathways by Kruskal-Wallis test across sample groups (BH-adjusted).

Pathway	p (KW)	p adj (BH)
D-galacturonate degradation I	0.0160	0.237
taxadiene biosynthesis (engineered)	0.0163	0.237
superpathway of L-aspartate and L-asparagine biosynthesis	0.0923	0.335
L-histidine degradation I	0.0367	0.335
methyl-coenzyme M oxidation to CO2	0.0892	0.335
GDP-mannose biosynthesis	0.0785	0.335
palmitate biosynthesis (type II fatty acid synthase)	0.0707	0.335
5'-deoxyadenosine degradation II	0.0770	0.335
L-citrulline biosynthesis	0.1350	0.434
homolactic fermentation	0.6060	0.679

```
top_paths <- kw_res %>% slice_head(n = 10) %>% pull(pathway)

path_long %>%
  filter(pathway %in% top_paths) %>%
  mutate(pathway = factor(pathway, levels = top_paths),
         label = str_remove(pathway, "[^:]+: ") %>% str_trunc(45)) %>%
  mutate(label = factor(label, levels = unique(label))) %>%
  ggplot(aes(x = type, y = cpm, colour = type)) +
  geom_boxplot(outlier.shape = NA, width = 0.5) +
  geom_jitter(width = 0.2, size = 0.7, alpha = 0.6) +
  facet_wrap(~label, scales = "free_y", ncol = 3) +
  scale_colour_manual(values = c("CRC" = "#E41A1C", "Healthy" = "#4DAF4A", "Paired" =
  ↪ "#377EB8")) +
  labs(x = NULL, y = "Pathway abundance (CPM)",
       title = "Figure -- Top 15 pathways by Kruskal-Wallis (BH-adjusted)") +
  theme_classic() +
  theme(legend.position = "none", strip.text = element_text(size = 7),
       axis.text.x = element_text(size = 8))
```

Figure -- Top 15 pathways by Kruskal-Wallis (BH-adjusted)

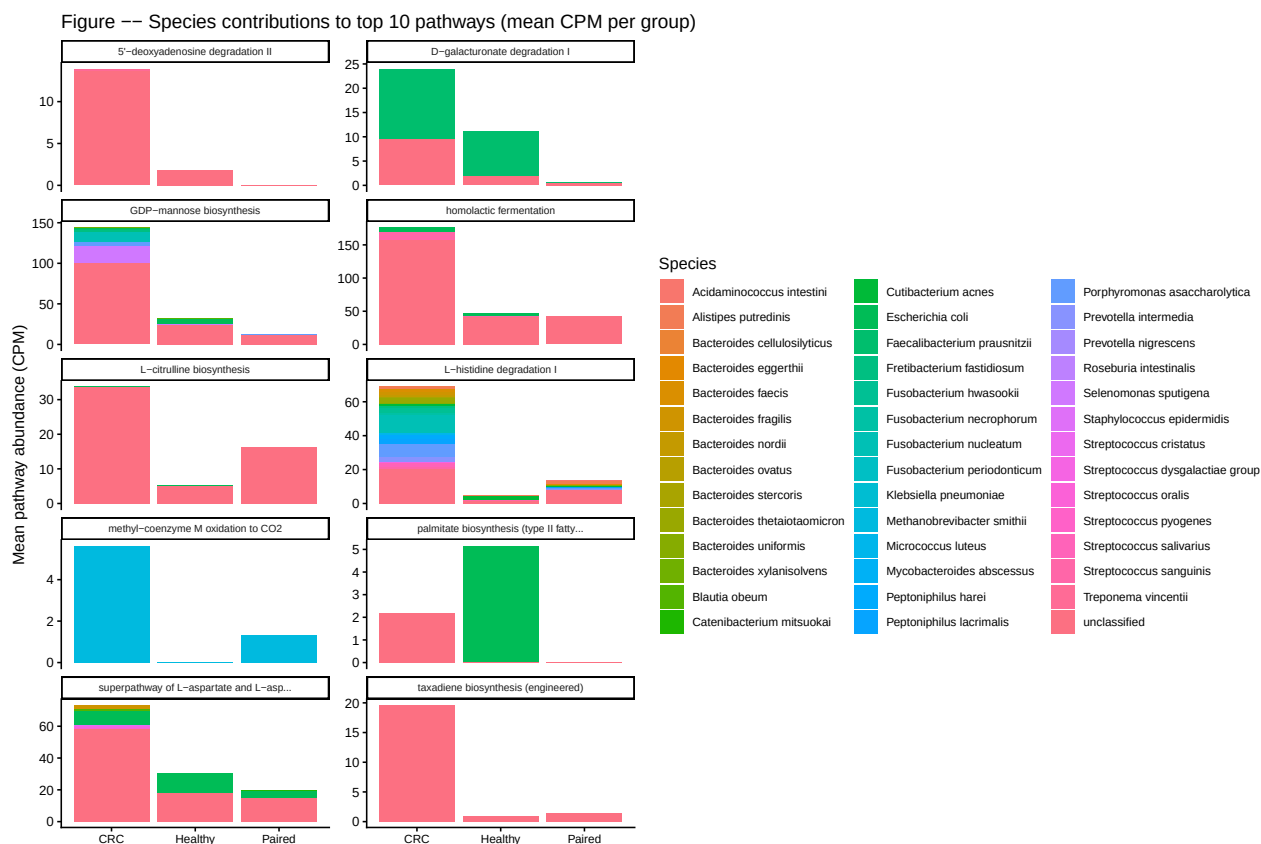


- No statistically significantly different pathways. however, good to visualize still

```
# Parse stratified rows (PATHWAY/g__Genus.s__species")
strat_long <- path_strat %>%
  filter(grepl("\\|", pathway_species)) %>%
  separate(pathway_species, into = c("pathway", "species"), sep = "\\|") %>%
  pivot_longer(-c(pathway, species), names_to = "sample", values_to = "cpm") %>%
  left_join(meta_h3, by = "sample") %>%
  filter(!is.na(type)) %>%
  mutate(species = str_remove(species, ".s__") %>% str_replace_all("_", " "))

strat_long %>%
  filter(pathway %in% top_paths[1:10]) %>%
  mutate(label = str_remove(pathway, "^[:]+: ") %>% str_trunc(40)) %>%
  group_by(label, species, type) %>%
  summarise(mean_cpm = mean(cpm), .groups = "drop") %>%
  ggplot(aes(x = type, y = mean_cpm, fill = species)) +
  geom_col(position = "stack") +
  facet_wrap(~label, scales = "free_y", ncol = 2) +
  labs(x = NULL, y = "Mean pathway abundance (CPM)", fill = "Species",
       title = "Figure -- Species contributions to top 10 pathways (mean CPM per group)")
  ↵ +
```

```
theme_classic() +
theme(strip.text = element_text(size = 7), axis.text.x = element_text(size = 8),
      legend.text = element_text(size = 8))
```



Stratified HUMANN pathway analysis shows that several CRC-enriched pathways were driven by a limited subset of bacterial taxa rather than broad community-wide contributions. Multiple pathways, including GDP-mannose biosynthesis, homolactic fermentation, and L-citrulline biosynthesis, shows visibly higher mean abundance in CRC biopsies, with dominant contributions from a small number of taxa, including several oral-associated anaerobes previously implicated in colorectal cancer. In contrast, healthy and paired samples generally exhibited lower pathway abundance and more evenly distributed species contributions.

Discussion

- hard to get statistically significant data from these kind of samples
- want to be robust by looking at everything (not just picking a couple like the authors did)
- similar to the authors, i've also run SAMSA2, a microbial analysis pipeline that uses DIAMOND, against just the *b.fragilis* genome, which seems to give insight, but with the high mapping %

Conclusion

[To be added]

References

References are hyperlinked throughout the document. Below is the full list in order of appearance.

1. Pereira-Marques J et al. (2024). Metatranscriptomic profiling of low-biomass samples: implications for the study of host-microbiome interactions. *PMC*. <https://pmc.ncbi.nlm.nih.gov/articles/PMC10913719/>
2. Franzosa EA et al. (2018). Species-level functional profiling of metagenomes and metatranscriptomes. *Nature Methods*. <https://pmc.ncbi.nlm.nih.gov/articles/PMC8096432/>
3. Kvich L et al. (2024). [Full citation to be added]

AI acknowledgement:

scripts created partially or wholly using Claude 3.5 has the line:

Credit: This file was partially generated using using Claude 3.5 Sonnet by Anthropic - AH